PropOS Phase 1 项目开发日程计划

> 版本: v1.3
>
> 制定日期: 2026-04-05（v1.3 更新日期: 2026-04-08）
>
> 计划开始日期: 2026-04-08（周三）
>
> 计划上线日期: 2026-08-21（周五）
>
> 开发模式: Copilot Agent 主导（~85% 零手写代码）+ 人工基础设施配置
>
> 每日投入: 8 小时
>
> 总工期: 20 周 / 98 个工作日
>
> 依据文档: PRD v1.8 / ARCH v1.4 / data_model v1.4

Copilot 协作规范

> 本节说明如何在 VS Code 中高效使用 Copilot（Claude 模型）推进每日开发，避免 AI 偏离架构设计与数据模型。
>
> **v1.3 升级说明**：整个项目已具备 ~85% 零手写代码能力。推荐优先使用 Agent 模式，按模块整体编排；仅在精细化调试或单层补丁时退回单提示词模式。

工作流选择

模式 *A*：*Agent* 整模块编排（推荐，优先使用）

适合完整模块实现（如"实现 M1 资产模块后端四层"）。Copilot 将自动按 9 步依序生成代码、运行测试、输出完成报告。

1. 打开 VS Code **Copilot Chat（Agent 模式）**
2. 在 Agent 选择器中选择 `**PropOS Feature Builder**`
3. 用一句话描述要实现的模块，例如：

```
实现资产模块 M1 的后端四层：Building + Floor + Unit + Renovation
```

1. Agent 会自动读取所有相关架构文档，按 Step 1–9 依序输出，每步完成后更新进度
2. 完成后，立即用 `**PropOS Compliance Reviewer**` 审查（见下方）

模式 *B*：单层精细提示词（次选，用于补丁／调试）

适合只修改某一层、修复单个 Bug，或 Agent 模式遗漏了某个细节时使用。

1. 在 Chat 输入框输入 `/` 选择对应斜杠命令（见下表）
2. 粘贴当日 `> 💬 Copilot 提示语` 引用块中的任务描述
3. 末尾附带标注的 `@file:` 引用（Agent 模式下自动解析）

可用资源（Agent + 提示词模板）

专用 *Agent*（模式 *A*）

Agent	调用方式	职责
`PropOS Feature Builder`	Agent 选择器	端到端模块实现（Step 1 迁移 → Step 9 UI），自动编排 9 层
`PropOS Compliance Reviewer`	Agent 选择器	只读合规审查，返回逐层违规报告，每个 **Milestone** 后必须执行一次

单层提示词模板（模式 *B*，斜杠命令）

模板文件	斜杠命令	适用任务类型
`pure-dart-package.prompt.md`	`/pure-dart-package`	纯计算包（`rent_escalation_engine`、`kpi_scorer`）
`backend-module.prompt.md`	`/backend-module`	后端四层（Model + Repository + Service + Controller）
`uniapp-page.prompt.md`	`/uniapp-page`	uni-app 页面（类型 + API + Store + 页面）
`admin-view.prompt.md`	`/admin-view`	admin 视图（类型 + API + Store + 视图）
`security-and-test.prompt.md`	`/security-and-test`	安全审查 + 性能测试 + 集成测试

仍需人工操作的步骤（不可绕过）

操作	说明
基础设施搭建	PostgreSQL 启动、`.env` 环境变量填写、ODA File Converter 安装
执行数据库迁移	`psql -f migration.sql` 需连接真实 DB
UAT 设备验收	真实用户在设备上的业务流程验收

> Copilot Agent 可运行 `dart test`、`npm install` 等终端命令，但无法连接用户本地数据库或访问外部服务。

通用防偏离技巧（由 Compliance Reviewer 自动执行，此处供人工复核参考）

常见偏离点	检查命令
引入了 ORM（如 `drift` / `sqflite` ）	`grep -r "drift\
Store 直接使用 fetch/axios	`grep -rn "fetch(\
页面硬编码了 API 路径	`grep -rn '"/api/' app/src/pages/ admin/src/views/`
Controller 直接 return Response	`grep -r "return Response\." backend/lib/modules/`
SQL 字符串拼接（注入风险）	`grep -rn '"\$' backend/lib/modules/*/repositories/`

总体排期概览

阶段	周次	日历区间	核心交付	工作日
Phase 0：基础搭建	W1–W2	4/8–4/17	核心 Package + 后端框架 + Auth	8 天
Phase 1：M1 资产	W3–W5	4/20–5/8	资产台账 + CAD 导入 + SVG 热区图	15 天
Phase 2：M2 合同	W6–W9	5/11–6/5	合同生命周期 + 递增规则 + WALE	20 天
Phase 3：M4 工单	W10–W11	6/8–6/19	工单流转 + 移动端报修 + 推送	10 天
Phase 4：M3 财务	W12–W15	6/22–7/17	账单 + NOI 看板 + KPI 仪表盘	20 天
Phase 5：M5 穿透	W16–W17	7/20–7/31	子租赁 + 外部填报门户 + 审核流	10 天
Phase 6：集成测试	W18–W19	8/3–8/14	模块联动 + 数据初始化 + 回归	10 天
Phase 7：UAT 上线	W20	8/17–8/21	用户验收 + 正式 Go-live	5 天

模块开发顺序依据：

- M1（资产）是数据底座，其他模块均依赖单元/楼层实体
- M2（合同）是业务主轴，M3 账单依赖 M2 递增规则，M5 穿透依赖 M2 主合同
- M4（工单）相对独立，M3 NOI 支出侧依赖 M4 维修成本，故先 M4 再 M3
- M3（财务）依赖 M1/M2/M4 全部数据源（KPI 10 指标跨模块聚合）
- M5（二房东穿透）依赖 M1 单元、M2 主合同，最后实现

里程碑一览

> 每个里程碑完成后，必须用 **`PropOS Compliance Reviewer` Agent** 对该阶段所有新增模块执行一次合规审查，确认无违规后方可进入下一阶段。

里程碑	完成日期	核心验证标准	合规审查范围
M0 基础就绪	2026-04-17	两个核心 Package 全测试通过；后端启动成功；前端登录页联调跑通	`backend/lib/core/` + `backend/lib/modules/auth/` + `app/src/` + `admin/src/`
M1 资产上线	2026-05-08	639 套 Excel 导入正常；楼层 SVG 热区图渲染正确；资产 Dashboard 三业态数据展示	`backend/lib/modules/assets/` + `app/src/pages/assets/` + `admin/src/views/assets/`
M2 合同上线	2026-06-05	合同状态机全流程；租金递增 6 种类型 + 混合分段；WALE 误差 < 0.01 年	`backend/lib/modules/contracts/` + `app/src/pages/contracts/` + `admin/src/views/contracts/`
M4 工单上线	2026-06-19	移动端报修全链路；FCM 推送验证；成本接口预留	`backend/lib/modules/work_orders/` + `app/src/pages/workorders/` + `admin/src/views/workorders/`
M3 财务上线	2026-07-17	账单生成 < 30 秒；NOI 三业态拆分正确；KPI 2 套方案打分与手工一致	`backend/lib/modules/finance/` + `admin/src/views/finance/`
M5 穿透上线	2026-07-31	外部门户可独立访问；审核流完整；行级隔离安全验证通过	`backend/lib/modules/subleases/` + `admin/src/views/subleases/`
集成完成	2026-08-14	全 PRD 验收项 Pass；50 并发压测达标；实际数据导入完成	全模块交叉合规扫描
正式上线	**2026-08-21**	**PropOS Phase 1 全模块生产环境就绪**	—

Phase 0：项目基础搭建

> 时间：2026-04-08（W1）— 2026-04-17（W2），共 8 个工作日

第 1 周（4/8 周三 — 4/10 周五）

Day 1 · 4月8日（周三）— `rent_escalation_engine` Package 骨架

- 创建 Monorepo 目录结构（`backend/packages/`、`app/`、`admin/`）
- `rent_escalation_engine/pubspec.yaml`（name: rent*escalation*engine，零外部依赖）
- `EscalationType` 枚举（6 种：fixedRate / fixedAmount / stepped / cpiLinked / everyNYears / postRenovation）
- `RentEscalationPhase` 数据类（阶段起止月份 + 递增参数 sealed union）
- `FixedRateRule` + `FixedAmountRule` 实现 + 单元测试 pass

> 💬 **Copilot 提示语（模板：`/pure-dart-package`）：**
>
> 在 `backend/packages/rent_escalation_engine/` 下创建包骨架：`pubspec.yaml`（dependencies 区块必须为空）、`EscalationType` 枚举（6种）、`RentEscalationPhase` sealed class、`FixedRateRule` / `FixedAmountRule` 子类及配套单元测试。**禁止引入任何外部依赖，不得 import flutter SDK，`dart test` 全绿才算完成。**

> 附： `@file:docs/backend/data_model.md` `@file:docs/ARCH.md`

Day 2 · 4月9日（周四）— `rent_escalation_engine` 复杂逻辑 + `kpi_scorer`

- `SteppedRule`（阶梯式分段表）+ `EveryNYearsRule`（每 N 年涨幅）实现
- `CpiLinkedRule`（CPI 年度挂钩）+ `PostRenovationRule`（免租结束后基准价）实现
- 混合分段 `RentCalculator.compute(phases, targetDate) → Money` 聚合逻辑
- `kpi_scorer/` Package： `KpiMetric`（含满分/及格/不及格阈值）+ `KpiScorer.score(metric, actual) → double`（线性插值）

> 💬 **Copilot 提示语**（模板： `/pure-dart-package` ）：
>
> 续 Day 1，完成剩余 4 个规则子类 + `RentCalculator.compute(phases, targetDate)` 多阶段聚合；另在 `backend/packages/kpi_scorer/` 创建 `KpiMetric` 和 `KpiScorer.score()`（线性插值，值域 [0,100] clamp，反向指标 direction:negative 时插值翻转）。**两个包均保持零外部依赖。**
>
> 附： `@file:docs/backend/data_model.md`（kpi*scheme*metrics.direction 字段）

Day 3 · 4月10日（周五）— 两个 Package 全覆盖测试

- `rent_escalation_test.dart`：固定比例 / 固定金额 / 阶梯 / 每 N 年 / CPI / 混合分段，每类型 ≥ 3 个用例
- `kpi_scorer_test.dart`：满分边界 / 线性插值中间值 / 零分边界 / 权重汇总 4 类用例
- `dart test` 全绿通过，Package 完工，可供后端 path 依赖引用（前端不直接引用，通过 API 调用）

> 💬 **Copilot 提示语**（模板： `/pure-dart-package` ）：
>
> 补齐两个包的单元测试：租金递增每种规则 ≥3 用例，**必须包含跨类型混合分段场景**（如"第1~2年固定比例+第3年CPI挂钩"）；KPI 覆盖正/反向指标各一组的满分/中间/零分边界。只用 `package:test`，不引入 mock 框架， `dart test` 全绿方可停止。

第 2 周（4/13 周一 — 4/17 周五）

Day 4 · 4月13日（周一）— 后端项目初始化

- `backend/pubspec.yaml`（依赖：shelf, shelf*router, postgres, dart*jsonwebtoken, freezed, json*serializable, build*runner, crypto, path）
- `bin/server.dart` 启动入口（Shelf Pipeline 组装、 `APP_PORT` 监听）
- `app_config.dart`：从 `Platform.environment` 读取 6 个必填变量，任一缺失时 `throw StateError` 输出明确错误
- `database.dart`：解析 `DATABASE_URL`，初始化 PostgreSQL 连接池

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 初始化 `backend/` 项目。 `app_config.dart` 从 `Platform.environment` 读取 6 个必填变量（ `DATABASE_URL` / `JWT_SECRET` / `JWT_EXPIRES_IN_HOURS` / `FILE_STORAGE_PATH` / `ENCRYPTION_KEY` / `APP_PORT` ），任一缺失 `throw StateError('缺少环境变量: VAR_NAME')` **——不得写成 Dart const**。 `database.dart` 连接池不得在日志中输出密码。 `bin/server.dart` 组装 Pipeline 顺序： `logMiddleware → rateLimitMiddleware → authMiddleware → rbacMiddleware` 。
>
> 附： `@file:docs/ARCH.md`（后端启动环境变量表、第2节目录结构）

Day 5 · 4月14日（周二）— 后端核心中间件

- `app_exception.dart`（`AppException(code, message, statusCode)` 基类）
- `error_handler.dart`（全局 `try/catch` → `{"error":{"code":"...","message":"..."}}` HTTP 响应）
- `request_context.dart`（注入 userId, role, subLandlordScope 到 Request 扩展）
- `auth_middleware.dart`（Bearer Token 解析，JWT 验证，注入 RequestContext）
- `pagination.dart`（解析 page/pageSize，构建分页 meta 响应包装工具）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 在 `backend/lib/core/` 下实现核心基础设施。**`error_handler.dart` 是全局唯一 HTTP 响应出口**，Controller/Service 只抛 `AppException`，绝不直接 `return Response`；错误 code 用 `SCREAMING_SNAKE_CASE`。分页 `pageSize` 默认 20、最大 100。`auth_middleware.dart` JWT 验证失败统一返回 401，不泄露解码细节，算法白名单为 `['HS256']`。
>
> 附：`@file:docs/ARCH.md`（API协议约定：响应信封格式、分页约定）

Day 6 · 4月15日（周三）— Auth 模块后端

- `user.dart`（@freezed）+ `users` 表迁移 SQL
- `user_repository.dart`（原生 SQL：findByUsername, findById, create, updateRole）
- `token_service.dart`（JWT 签发：sub + role + subLandlordScope + exp；验证：算法固定 HS256）
- `auth_service.dart`（登录：bcrypt 密码校验，签发 access + refresh token；刷新：验证 refresh token 后重新签发）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 在 `backend/lib/modules/auth/` 下实现认证四层。安全硬约束：**JWT 算法固定`HS256`，验证时`allowedAlgorithms: ['HS256']`，禁止 alg:none**；SQL 全部参数化，禁止字符串拼接；`password` 字段不得出现在任何 JSON 响应；bcrypt cost ≥ 12；登录失败累计超限返回 `ACCOUNT_LOCKED`（423）。
>
> 附：`@file:docs/backend/data_model.md`（users 表）`@file:docs/ARCH.md`

Day 7 · 4月16日（周四）— RBAC + 审计 + Auth 路由

- `rbac_middleware.dart`（按端点路径 + HTTP 方法配置角色白名单矩阵，权限不足返回 403）
- `audit_middleware.dart`（写 `audit_logs` 表：操作用户、端点、before/after JSON、时间戳）
- `auth_controller.dart`（`POST /api/auth/login`、`POST /api/auth/refresh`）
- `router/` 挂载，`bin/server.dart` 本地启动，curl 验证登录接口

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 RBAC 和审计。`rbac_middleware.dart` 权限矩阵用 `Map<String, Map<String, Set<String>>>`（路径→方法→允许角色），**禁止散落 if-else**；无权限返回 403 + `FORBIDDEN` code。`audit_middleware.dart` 必须覆盖4类高风险操作（合同变更/账单核销/权限变更/二房东提交），before/after 均为完整 JSON 非 null。Controller 只调用 Service，不含业务逻辑。
>
> 附：`@file:docs/ARCH.md`（RBAC章节）`@file:docs/backend/data_model.md`（audit_logs 表）

Day 8 · 4月17日（周五）— 前端项目初始化 + Auth 前端

- uni-app 端：`cd app && npm install`，核心依赖：`wot-design-uni`、`luch-request`、`pinia`、`dayjs`
- admin 端：`cd admin && npm install`，核心依赖：`element-plus`、`axios`、`pinia`、`vue-router`、`dayjs`
- 骨架目录：`app/src/`（api, stores, types, constants, pages）+ `admin/src/`（api, stores, types, constants, views, router）
- `app/src/api/client.ts`（luch-request 封装 + JWT Bearer 拦截器 + 401 刷新）
- `admin/src/api/client.ts`（axios 封装 + JWT Bearer 拦截器 + 401 refresh subscriber queue）
- `useAuthStore`（Pinia setup 风格）+ 登录页 UI（app + admin 各一套）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 初始化前端项目并完成 Auth 全链路。必须遵守：uni-app 端所有颜色通过 CSS 变量（`--color-success` 等），admin 端通过 Element Plus `type` 属性；`useAuthStore` 使用 `defineStore(id, setup)` 风格，state 含 `token / user / loading / error`；`api/client.ts` 401 刷新不能死循环（用标志位 + subscriber queue 防止重复刷新）；页面只访问 store，不内联 HTTP 请求。
>
> 附：`@file:docs/ARCH.md`（前端分层规则）`@file:.github/copilot-instructions.md`
>
> **Milestone 0**：`dart test` 两个核心 Package 全绿；`dart run bin/server.dart` 后端启动成功；前端登录页与后端 `POST /api/auth/login` 端到端联调成功。

Phase 1：M1 资产与空间可视化

> 时间：2026-04-20（W3）— 2026-05-08（W5），共 15 个工作日

第 3 周（4/20 — 4/24）· M1 后端 CRUD

Day 9 · 4月20日（周一）— M1 API Contract + 路由骨架

- 输出 `docs/api/m1_assets.md`（buildings / floors / units / renovations 全部端点，含请求参数、响应 JSON、枚举表、error code，不转 PDF）
- 后端全部 M1 路由骨架：Controller 空实现，返回固定 mock JSON，服务可运行供前端提前对接结构

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 为 M1 输出 API 文档 `docs/api/m1_assets.md`，然后在 `backend/lib/modules/assets/controllers/` 下创建 Building/Floor/Unit/Renovation 四个 Controller 骨架（返回固定 mock JSON），使服务可运行。API 文档中响应格式必须严格使用 `{"data":...,"meta":{...}}` 信封，错误码用 `SCREAMING_SNAKE_CASE`，枚举值与 data_model.md 一致。骨架阶段暂不实现业务逻辑，但路由路径必须与最终规格一致。
>
> 附：`@file:docs/backend/data_model.md`（buildings/floors/units 表）`@file:docs/ARCH.md`

Day 10 · 4月21日（周二）— Building + Floor 后端

- `building.dart` / `floor.dart` （@freezed）+ 数据迁移 SQL（参照 data_model.md）
- `BuildingRepository` （list, getById, create, update；原生 SQL，LIMIT/OFFSET 分页）
- `FloorRepository` （含 svgPath 字段； `listByBuilding` ）
- `BuildingService` + `FloorService` （业务校验层）
- `BuildingController` （ `GET /api/buildings` ， `GET/PUT /api/buildings/:id` ）+ `FloorController` （ `GET /api/buildings/:id/floors` ）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 在 `backend/lib/modules/assets/` 下实现 Building + Floor 完整四层。数据库字段名用 `snake_case` （与 data_model.md 精确对齐），Repository 所有 SQL 使用参数化查询。 `floor.dart` 包含 `svgPath` （ `String?` ）字段，对应文件路径格式 `floors/{buildingId}/{floorId}.svg` 。Controller 返回标准信封格式，分页方法附带 `meta` 。**禁止 ORM，禁止在 Controller 写业务判断。**
>
> 附： `@file:docs/backend/data_model.md` （buildings/floors 表） `@file:docs/ARCH.md`

Day 11 · 4月22日（周三）— Unit 后端（三业态差异化字段）

- `unit.dart` （@freezed，含 `propertyTypeDetails` JSONB 字段：OfficeDetails / RetailDetails / ApartmentDetails）
- `UnitRepository` （CRUD + 多条件查询：buildingId / propertyType / status； `getFloorHeatmap` — 返回楼层所有单元坐标 + 状态）
- `UnitService` （当前状态计算：Leased / Vacant / ExpiringSoon≤90天 / NonLeasable；单元不重叠在租校验）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 实现 `unit.dart` （@freezed）和 `UnitRepository` / `UnitService` 。 `propertyTypeDetails` 字段为 JSONB，用 sealed class（ `OfficeDetails` / `RetailDetails` / `ApartmentDetails` ）区分三业态扩展信息。 `getFloorHeatmap` 方法返回楼层所有单元的坐标多边形 + 当前状态（用于前端热区图叠加）。**状态计算逻辑在 Service 层**，Repository 只做数据存取；同一单元时段重叠校验用 SQL `EXCLUDE` 约束或应用层 OVERLAP 检查。
>
> 附： `@file:docs/backend/data_model.md` （units 表+扩展字段） `@file:docs/ARCH.md`

Day 12 · 4月23日（周四）— Unit Controller + RenovationRecord

- `UnitController` （ `GET /api/units` ， `GET/POST/PUT /api/units/:id` ， `GET /api/floors/:id/heatmap` ）
- `renovation_record.dart` （@freezed）+ `RenovationRepository` + `RenovationService`
- `RenovationController` （ `GET /api/units/:id/renovations` ， `POST /api/units/:id/renovations` ，照片上传）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 完成 Unit Controller 和 Renovation 模块。改造照片存储路径为 `renovations/{recordId}/{index}.jpg` （通过 `GET /api/files/*` 代理访问，不暴露存储路径）。Controller 只做参数解析和调用 Service，不写任

> 何业务判断。`**GET /api/floors/:id/heatmap**` 端点需经过 **RBAC** 验证，仅返回坐标和状态，不返回合同商业细节。
>
> 附：`@file:docs/backend/data_model.md`（renovation_records 表）`@file:.github/copilot-instructions.md`（文件存储约定）

Day 13 · 4月24日（周五）— Excel 批量导入后端

- `unit_import_service.dart`：解析三种 Excel 模板（写字楼 / 商铺 / 公寓字段映射），数据校验（单元号唯一、面积正数、业态枚举合法），批量 INSERT（事务包裹）
- `POST /api/units/import`（multipart/form-data，响应：成功条数 + 失败行列表）
- 本地用 20 条/业态样本数据测试导入正确性

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 在 `backend/lib/modules/assets/services/unit_import_service.dart` 实现 Excel 批量导入。用事务包裹批量 INSERT，任意一行校验失败整批回滚并返回详细错误行列表。业态枚举值必须与 data*model.md* 中 `*property*type` 枚举精确匹配（`office` / `retail` / `apartment`）。**引入 import*batches*` 表记录每次导入批次**（*dry*run 模式先不写库，只返回校验结果）。
>
> 附：`@file:docs/backend/data_model.md`（units 表、import_batches 表）

第 4 周（4/27 — 5/1）· M1 后端 CAD 导入 + 前端 Domain/Data

Day 14 · 4月27日（周一）— CAD 导入后端（难点）

- 安装配置 ODA File Converter（DWG→DXF）+ `ezdxf[draw]`（Python，DXF→SVG），验证两步转换链路可用
- `cad_import_service.dart`：接收 `.dwg` 文件 → `Process.run` 调度 ODA File Converter（DWG→DXF）→ `Process.run` 调度 `ezdxf draw`（DXF→SVG）→ SVG 输出写入 `FILE_STORAGE_PATH/floors/{buildingId}/{floorId}.svg`
- `PUT /api/floors/:id/cad-upload`（multipart 大文件，异步任务，上传完成后返回 SVG 预览 URL）
- `GET /api/files/*`（鉴权后返回本地文件流，不直接暴露存储路径）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 `cad_import_service.dart`，通过两步 `Process.run` 链路（ODA Converter→DXF，ezdxf→SVG）进行 CAD 转换。SVG 输出路径严格遵守 `floors/{buildingId}/{floorId}.svg` 格式（UUID，非业务号）；**转换为异步任务**，上传端点立即返回任务 ID，通过轮询或回调获取结果。`GET /api/files/*` 必须验证 JWT，文件路径必须在 `FILE_STORAGE_PATH` 目录沙箱内（防路径穿越），用 `path.canonicalize()` + 前缀检查。
>
> 附：`@file:.github/copilot-instructions.md`（文件存储约定）`@file:docs/ARCH.md`

Day 15 · 4月28日（周二）— M1 前端类型定义 + API 函数

- TypeScript 接口：`Unit`，`Building`，`Floor`，`RenovationRecord`（放 `app/src/types/` 和 `admin/src/types/`）

- API 函数：`unitApi`，`buildingApi`，`floorApi`，`renovationApi`（放 `app/src/api/modules/` 和 `admin/src/api/modules/`）
- 路径常量：`API_PATHS.units`，`API_PATHS.buildings` 等（放 `src/constants/api_paths.ts`）

> 💬 **Copilot 提示语**（模板：`/uniapp-page`）：
>
> 在 `app/src/types/` 下创建 M1 TypeScript 接口定义。`Unit.status` 枚举值（`leased` / `vacant` / `expiring_soon` / `non_leasable`）与后端 `snake_case` 保持一致；在 `app/src/api/modules/asset.ts` 下封装 API 函数，使用 `apiGet` / `apiPost`（来自 `@/api/client`），路径来自 `@/constants/api_paths`（禁止硬编码）；分页请求统一传 `{ page, pageSize }` 参数。
>
> 附：`@file:docs/backend/data_model.md`（buildings/floors/units 表）`@file:docs/ARCH.md`

Day 16 · 4月29日（周三）— M1 前端 Pinia Store 层

- `useAssetOverviewStore`（加载三业态汇总：总套数 / 已租套数 / 空置套数 / 出租率）
- `useBuildingStore`（列表 + 筛选：按业态 / 状态 / 翻页）
- `useUnitStore`（列表 + 多条件筛选：propertyType / status / buildingId / 翻页）
- Mock 模式：通过 `VITE_USE_MOCK=true` 环境变量切换，Store 内条件返回模拟数据

> 💬 **Copilot 提示语**（模板：`/uniapp-page`）：
>
> 在 `app/src/stores/` 下实现 M1 Pinia Store。使用 `defineStore(id, setup)` 风格；state 固定字段 `list` / `item` / `loading` / `error` / `meta`；action 调用 `@/api/modules/asset` 的 API 函数（禁止在 Store 中直接写 HTTP 请求）；异常捕获统一 `catch (e) { error.value = e instanceof ApiError ? e.message : '操作失败，请重试' }`；Mock 数据覆盖 `leased` / `vacant` / `expiring_soon` / `non_leasable` 四种状态各至少一条。
>
> 附：`@file:docs/ARCH.md`（前端分层规则）`@file:.github/copilot-instructions.md`

Day 17 · 4月30日（周四）— M1 Store 单元测试 + admin Store

- uni-app Store 单元测试（Vitest，mock API 函数，覆盖 loading → loaded / error 场景）
- `admin/src/stores/` 下同步实现 `useAssetOverviewStore`、`useBuildingStore`、`useUnitStore`（逻辑与 app 端一致，HTTP 客户端为 Axios）
- `admin/src/api/modules/asset.ts` — admin 端 API 函数
- Store 超过 200 行时按子领域拆分（如 `useUnitListStore` + `useUnitFilterStore`）

> 💬 **Copilot 提示语**（模板：`/admin-view`）：
>
> 在 `admin/src/stores/` 下实现 M1 Pinia Store。使用 `defineStore(id, setup)` 风格，state 固定字段 `list` / `item` / `loading` / `error` / `meta`；action 调用 `@/api/modules/asset` 的 API 函数（禁止直接写 axios 调用）；同步编写 Vitest 单元测试：mock API 函数，覆盖 loading→loaded、loading→error 两个场景。Store 超 200 行时按职责拆分。
>
> 附：`@file:docs/ARCH.md`（前端分层规则、文件复杂度超限拆分策略）

Day 18 · 5月1日（周五）— M1 UI — 资产概览 Dashboard

- uni-app `AssetOverviewPage`：三列卡片（写字楼 / 商铺 / 公寓），每列展示：总套数、已租套数、空置套数、出租率进度条
- `BuildingListPage` + `BuildingCard` 组件（楼栋名称、业态、GFA、出租率快速跳转楼层）
- admin `DashboardView`：Element Plus Card + Progress 组件，同步实现 PC 端资产概览
- 状态色严格使用 CSS 变量 / Element Plus type：`leased` → success（绿），`vacant` → danger（红），`expiring_soon` → warning（橙），`non_leasable` → info（灰）

> 💬 **Copilot 提示语**（模板：`/uniapp-page`）：
>
> 在 `app/src/pages/assets/` 下实现资产 Dashboard。严格约束：**所有状态色通过 CSS 变量**（`--color-success` / `--color-danger` / `--color-warning` / `--color-neutral`），禁止内联 `style="color: green"` 或硬编码色值；页面超 250 行时将业态卡片提取到 `components/PropertyTypeCard.vue`；页面只使用 Store 的 state/action，不内联 HTTP 请求；进度条使用 `wot-design-uni` 的 `wd-progress` 组件。
>
> 附：`@file:.github/copilot-instructions.md`（UI色彩规范、状态色语义映射）

第 5 周（5/4 — 5/8）· M1 前端 楼层热区图 + 联调

Day 19 · 5月4日（周一）— 楼层平面图 SVG 渲染（难点）

- uni-app `FloorMapPage`：通过 `<image>` 或内嵌 `<web-view>` 加载楼层 SVG 文件（通过 `/api/files/floors/...` 代理）
- admin `FloorMapView`：使用 SVG 内联 + `<svg>` 元素叠加半透明状态色块多边形（coord 来自 API `heatmap` 端点）
- 楼层切换 Tab / Dropdown，按业态显示/隐藏开关，SVG 支持手势缩放平移（CSS `transform` + touch 事件 / admin 端鼠标滚轮）

> 💬 **Copilot 提示语**（模板：`/uniapp-page`）：
>
> 实现 `FloorMapPage`：加载 SVG（URL 通过 `API_PATHS.fileProxy(path)` 拼接，含 Authorization 头），支持手势缩放平移；SVG 叠加热区多边形，颜色来自 CSS 变量（**不硬编码**），透明度 0.4。**楼层切换和业态过滤逻辑在`useFloorMapStore`**，不在组件中维护状态；`FloorMapPage` 超 250 行时将 SVG 叠加层提取为 `components/FloorHeatmapOverlay.vue`。
>
> 附：`@file:.github/copilot-instructions.md`（UI色彩、状态色）`@file:docs/ARCH.md`（平台能力矩阵）

Day 20 · 5月5日（周二）— 热区交互 + 单元详情

- 热区点击识别：SVG 元素 click 事件 + 坐标命中判断 → 查找命中 unitId → 跳转 `UnitDetailPage`
- `UnitDetailPage`：基本信息（面积 / 楼层 / 朝向 / 装修状态）+ 三业态差异化扩展字段 + 当前合同摘要占位（待 M2 联动）
- `RenovationHistory` 组件：改造记录列表（改造类型 / 日期 / 造价）+ 照片网格

> 💬 **Copilot 提示语**（模板：`/uniapp-page`）：

> 实现热区点击交互和 `UnitDetailPage` 。点击 SVG 多边形通过元素 `@click` 事件 + `data-unit-id` 属性识别，命中后 `uni.navigateTo({ url: '/pages/assets/unit-detail?id=${unitId}' })` 跳转。 `UnitDetailPage` 中当前合同摘要区域目前为占位组件（ `ContractSummaryPlaceholder` ），待 M2 联动后回填。三业态差异化字段用 `v-if="unit.propertyType === 'office'"` 等条件渲染（admin 端同理）。
>
> 附： `@file:docs/ARCH.md` （前端路由） `@file:.github/copilot-instructions.md`

Day 21 · 5月6日（周三）— Excel 导入前端 + 资产台账导出

- uni-app `ImportPage` ：业态选择 Tabs + `uni.chooseFile` 选择 Excel + 上传进度 + 结果报告（成功条数 / 失败行明细）
- admin `UnitListView` （Element Plus Table，列：单元号 / 业态 / 面积 / 状态 / 当前租客，支持筛选 + 翻页）
- 导出按钮 → `GET /api/units/export` （后端生成 Excel，前端下载保存）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现 `ImportPage` （uni-app）和 `UnitListView` （admin）。上传进度用 `luch-request` 的 `onUploadProgress` 回调驱动 `wd-progress` （uni-app）或 `el-progress` （admin）；错误行用表格展示（列：行号/字段/错误原因）。翻页大小使用 `DEFAULT_PAGE_SIZE` （来自 `ui_constants.ts` ），禁止硬编码 `20` ；状态列颜色用 CSS 变量（uni-app）或 Element Plus Tag type（admin）；筛选条件变化通过 Store action 触发，不在组件中做业务判断。
>
> 附： `@file:.github/copilot-instructions.md` （常量管理规则、UI色彩规范）

Day 22 · 5月7日（周四）— M1 前后端联调

- 切换 `USE_MOCK=false` ，对接真实后端
- 修复数据格式差异（snake_case → camelCase 自动转换验证，日期 ISO 8601 解析，枚举映射不缺值）
- 真实 SVG 文件上传测试（测试用 .dwg → ODA File Converter→DXF → ezdxf→SVG 两步转换链路 → 前端渲染验证）

> 💬 **Copilot 提示语**（排查联调问题用）：
>
> 我在联调 M1 前后端，遇到以下问题：[描述具体报错或数据差异]。请对照 `@file:docs/ARCH.md` （API协议约定）和 `@file:docs/backend/data_model.md` ，帮我定位是后端响应格式不符（信封/字段名/枚举值）还是前端解析逻辑问题，给出最小修改方案。**不要引入新依赖，不要改变架构层次，只修复数据格式对齐问题。**

Day 23 · 5月8日（周五）— M1 缺陷修复 + 冒烟测试

- 按 PRD §七逐项核对：CAD 平面图展示、单元色块随合同状态联动（模拟切换状态验证）
- RBAC 验证：前线员工访问 `POST /api/units` 返回 403
- 记录 M1 → M2 联动遗留点（UnitDetailPage 当前合同摘要、楼层热区颜色驱动来源）

> 💬 **Copilot 提示语**（安全验证用）：
>
> 帮我验证 M1 RBAC 是否正确实现：前线员工（role: `frontline_staff` ）访问 `POST /api/units` 和 `DELETE /api/units/:id` 应返回 403；管理员（role: `admin` ）应200。请对照 `@file:docs/ARCH.md`

> RBAC 章节，检查 `rbac_middleware.dart` 的权限矩阵配置是否覆盖这些端点，并给出补充缺失权限规则的代码，保持矩阵结构不变，仅追加行。
>
> **Milestone 1**：639 套 Excel 样本导入正常；楼层 SVG 热区图在前端渲染正确；资产 Dashboard 三业态汇总数据展示。

Phase 2：M2 租务与合同管理

> 时间：2026-05-11（W6）— 2026-06-05（W9），共 20 个工作日

第 6 周（5/11 — 5/15）· M2 后端 Tenant + Contract

Day 24 · 5月11日（周一）— M2 API Contract + Tenant 后端

- 输出 `docs/api/m2_contracts.md`（tenants / contracts / rent-escalation / alerts / wale 全部端点）
- `tenant.dart`（@freezed，`idNumber` 字段标注 `// encrypted: AES-256`，API 响应默认后 4 位脱敏）
- `TenantRepository`（CRUD + 搜索；证件号存储前加密，取出后脱敏）
- `TenantService` + `TenantController`（`GET /api/tenants`，`GET/POST/PUT /api/tenants/:id`）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 输出 M2 API 文档并实现 Tenant 模块。关键安全约束：`tenant.dart` 的 `idNumber` 字段必须注释 `// encrypted: AES-256`；`TenantRepository` 存储前用 `ENCRYPTION_KEY` 进行 AES-256 加密，读取后 API 响应中仅返回后 4 位（格式 `****1234`）；恢复明文需记录审计日志（PIPL 合规）。`TenantController` 的搜索接口禁止模糊匹配加密列，改为匹配姓名/手机后4位明文索引字段。
>
> 附：`@file:docs/backend/data_model.md`（tenants 表）`@file:docs/ARCH.md`（架构约束第3条）

Day 25 · 5月12日（周二）— Contract 数据层

- `contract.dart`（@freezed，状态机枚举 + 免租期字段 + 付款周期 + 押金）
- `ContractRepository`（CRUD + 状态过滤 + 关联 Tenant/Unit JOIN 查询 + 分页）
- `contract_attachments` 表：`AttachmentRepository`（上传 → 存储 `contracts/{contractId}/{filename}` → 记录路径）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 `contract.dart`（@freezed）和 `ContractRepository`。**v1.7 变更**：合同与单元关系为 M:N，通过 `contract_units`（含 `billable_area` / `unit_price`）中间表关联，**禁止在 contracts 表添加 unit_id 外键**。合同包含 `taxInclusive`（bool）和 `applicableTaxRate`（Decimal?）字段，NOI 计算时统一使用不含税口径。合同终止类型 `terminationTypeEnum`：`normal_expiry` / `early_mutual` / `early_breach` / `early_force`。
>
> 附：`@file:docs/backend/data_model.md`（contracts/contract_units 表）`@file:docs/ARCH.md`

Day 26 · 5月13日（周三）— 合同状态机

- `ContractService` ：状态转换方法（draft→pending*sign, pending*sign→active, active→expiring_soon 自动判断, active→terminated, active→renewed）
- 业务校验：起租日 < 到期日；免租期 ≤ 合同期；同一单元同一时段不可重叠出租（SQL 时间段 OVERLAP 查询）
- 续签：新合同关联原合同 ID，形成合同链（ `predecessor_contract_id` ）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 在 `ContractService` 中实现合同状态机。状态流转逻辑：非法转换（如 `terminated→active` ）抛 `AppException('INVALID_STATUS_TRANSITION', ..., 422)` ；提前终止必须指定 `terminationType` 枚举（4种），终止后未来账单状态自动设为 `cancelled` （在同一事务中）；续签创建新合同时继承原合同递增规则（深拷贝 JSONB，不共享引用）；**所有状态变更必须写入`audit_logs`**（before/after 含完整合同 JSON）。重叠在租校验用 SQL `tstzrange` EXCLUDE 约束。
>
> 附： `@file:docs/backend/data_model.md` （contract*status* 枚举、*termination*type）

Day 27 · 5月14日（周四）— ContractController + 附件 + 递增规则持久化

- `ContractController` （ `GET/POST/PUT /api/contracts` ， `POST /api/contracts/:id/terminate` ， `POST /api/contracts/:id/renew` ， `GET /api/contracts/:id/attachments` ）
- `rent_escalation_service.dart` ：将 `List<RentEscalationPhase>` 序列化为 JSONB 存储；从库读取后反序列化，调用 `RentCalculator.compute()` 预算指定日期租金
- `GET /api/contracts/:id/rent-forecast` （按月返回全合同期预测租金数组）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 实现 ContractController 和递增规则持久化。 `rent_escalation_service.dart` 将 `List<RentEscalationPhase>` 序列化为 JSONB 写入 `contracts.escalation_rules` ，读取时反序列化后调用 `packages/rent_escalation_engine` 的 `RentCalculator.compute()` **——不允许在 Service 层内联递增计算逻辑，必须委托给独立 package。** `GET /api/contracts/:id/rent-forecast` 返回数组，每项含 `{month: "2026-05", amount: 12500.00, isExempt: false}` ，免租期月份 `isExempt: true` 。
>
> 附： `@file:docs/backend/data_model.md` （contracts.escalation_rules 字段） `@file:.github/copilot-instructions.md` （架构约束第5条）

Day 28 · 5月15日（周五）— WALE 服务

- `wale_service.dart` ：$WALE = \sum(剩余租期*i* *times* 年化租金i) / \sum(年化租金_i)$，调用 `RentCalculator` 获取各合同年化租金
- 支持 groupBy=overall / building / propertyType 三级
- `GET /api/wale` （支持 groupBy 参数）+ `GET /api/wale/trend?months=12` （历史 12 月 WALE 曲线数据）
- 用小样本合同数据手工验算对比，误差 < 0.001

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：

> 实现 `wale_service.dart` ，支持双口径（收入加权 + 面积加权）：$WALE{income} = \sum(remaining$i \times annualRent$i) / \sum(annualRent$i)$，$WALE{area} = \sum(remaining$i \times leasableArea$i) / \sum(leasableArea$i)$。剩余租期精确到天（用 UTC 日期差除以 365.25），**不依赖`DateTime.now()`**，从注入的 `Clock.now()` 获取当前时间。终止合同的剩余租期归零（不参与加权）。 `groupBy` 查询在 SQL 聚合层完成，不在 Dart 循环中遍历全量数据。
>
> 附： `@file:docs/ARCH.md` （WALE双口径公式） `@file:docs/backend/data_model.md` （contracts/contract_units 表）

第 7 周（5/18 — 5/22）· M2 后端 预警 + 前端类型/Store

Day 29 · 5月18日（周一）— 预警引擎后端

- `alert_service.dart` ：扫描合同生成到期预警（提前 90/60/30 天），扫描账单生成逾期预警（第 1/7/15 天未到账）
- `AlertRepository` （create, listByRecipient, markRead, markAllRead）
- `POST /api/scheduler/run-alerts` （手动触发，Phase 1 替代 cron；响应：新生成预警数量）
- `GET /api/alerts` （分页，过滤：type / isRead）+ `PUT /api/alerts/:id/read`

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 实现 `alert_service.dart` 。预警阈值（90/60/30天 到期；第1/7/15天逾期）定义在 `lib/shared/constants/business_rules.dart` （**禁止在 alert_service 中硬编码这些数字**）。扫描时批量插入，同一合同同一节点不得重复生成（用 UNIQUE 约束或 INSERT ON CONFLICT IGNORE）。 `job_runner.dart` 中注册定时任务钩子（Phase 1 手动触发），需具备失败重试和人工补偿能力（参考 `job_execution_log.dart` ）。
>
> 附： `@file:docs/backend/data_model.md` （alerts 表） `@file:.github/copilot-instructions.md` （常量管理规则）

Day 30 · 5月19日（周二）— M2 前端类型定义 + API 函数

- TypeScript 接口： `Tenant` ， `Contract` ， `ContractAttachment` ， `RentEscalationPhase` ， `Alert` ， `WaleData` ， `RentForecastItem` （放 `app/src/types/` 和 `admin/src/types/` ）
- API 函数： `tenantApi` ， `contractApi` ， `alertApi` ， `waleApi` ， `escalationApi` （放 `src/api/modules/` ）
- HTTP 封装（luch-request / axios + 分页 + ApiError 包装）+ Mock 模式（含 3 种附件格式样本）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 在 `app/src/types/` 和 `app/src/api/modules/` 下实现 M2 类型 + API 层。 `Contract` 接口包含 `taxInclusive` 和 `terminationType` 枚举字段； `RentEscalationPhase` 用 union type（6种子类型）与后端 `packages/rent_escalation_engine` 的类型一一对应（**前端不直接引用后端 package，通过 API 调用**）。API 层： `ApiError` 包装网络错误，不透传原始异常；Mock 数据包含 `active` / `expiring_soon` / `terminated` 三种合同状态。
>
> 附： `@file:docs/ARCH.md` （前端分层规则、package复用） `@file:docs/backend/data_model.md`

Day 31 · 5月20日（周三）— M2 BLoC

- `useTenantStore`（搜索 + 分页 + 详情加载）
- `useContractStore`（过滤：byStatus / byBuildingId / byPropertyType + 分页）
- `useAlertStore`（加载 + 标记已读 + 未读数 computed）+ `useWaleStore`（加载三级 WALE + 趋势）
- 全部 Store 单元测试通过（Vitest）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 在 `app/src/stores/` 和 `admin/src/stores/` 下实现 M2 Pinia Store。`useContractStore` 若超过 200 行，将"过滤条件管理"拆为独立 `useContractFilterStore`；`useAlertStore` 的未读数通过 `computed` 计算，不轮询。所有 Store 测试覆盖：①初始状态②加载成功③错误处理④过滤条件变更后重新加载。**禁止在 Store 中直接操作 UI（如弹 Toast）**，组件层通过 `watch` 或 `storeToRefs` 响应状态变化。
>
> 附：`@file:docs/ARCH.md`（文件复杂度超限拆分策略）`@file:.github/copilot-instructions.md`

Day 32 · 5月21日（周四）— M2 UI 租客详情 + 合同列表

- `TenantListPage`（搜索框 + 列表：名称 / 类型 / 手机尾号 / 信用评级）
- `TenantDetailPage`（全景画像：基本信息 / 租赁历史 Tab / 缴费信用 Tab / 工单记录 Tab 占位）
- `ContractListPage`（状态 Tab 筛选 + 合同卡片：单元号 / 租客名 / 到期日 / 月租金 / 状态色标）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现 `TenantListPage`、`TenantDetailPage` 和 `ContractListPage`。`TenantDetailPage` 工单 Tab 目前为占位组件，M4 完工后回填，**不要现在实现工单 List**。证件号显示时必须脱敏（仅末4位），Store 获取的数据中 `idNumber` 字段已为脱敏值，直接展示即可。信用评级（A/B/C）用 CSS 变量 / Element Plus Tag type 区分（A→success，B→warning，C→danger）。
>
> 附：`@file:.github/copilot-instructions.md`（UI色彩规范、状态色）`@file:docs/ARCH.md`

Day 33 · 5月22日（周五）— M2 UI 合同详情 + 操作 + 预警

- `ContractDetailPage`（合同详情 + 附件列表 + 操作区：终止/续签 确认弹窗）
- `ContractFormPage`（新建/编辑合同：单元选择 / 租客关联 / 免租期配置 / 付款周期）
- `AlertListPage`（按类型分组：到期预警 / 逾期预警 / 月度汇总；未读红标）
- `WaleDashboard` 组件（三业态 WALE 数值卡 + 12 月趋势折线图，admin 端用 ECharts）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现 `ContractDetailPage`、`ContractFormPage`、`AlertListPage`、`WaleDashboard`。合同状态色必须严格遵守：`active` →success，`expiring_soon` →warning，`terminated` →info，`overdue` →danger，**禁止任何硬编码颜色值**。`ContractFormPage` 的单元选择通过路由跳转或 Dialog（admin 端 `el-dialog`）。`WaleDashboard` 的 WALE 计算逻辑调用 Store 已计算值，**组件中不做数学计算**。删除确认弹窗必须双重确认（先弹确认框，用户输入合同编号后方可确认）。

> 附：`@file:.github/copilot-instructions.md`（UI色彩规范）`@file:docs/backend/data_model.md`（合同状态机）

第 8 周（5/25 — 5/29）· M2 前端 租金递增配置器（难点）

Day 34 · 5月25日（周一）— 配置器整体架构

- `RentEscalationConfigurator` 组件：多阶段动态列表，支持增删阶段按钮
- 每个阶段渲染 `EscalationPhaseCard` 子组件（阶段序号 + 类型下拉 + 动态参数区）
- `EscalationTypeSelector`：6 种类型下拉（uni-app 用 `wd-picker`，admin 用 `el-select`），选中后切换对应参数表单

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现租金递增配置器 `RentEscalationConfigurator`（admin 端 Vue 组件）。阶段列表用 `ref<EscalationPhase[]>` 本地状态管理；增删阶段只操作本地 state，保存时整体提交给 `useEscalationStore`。**禁止在组件内直接调用 HTTP**。`EscalationTypeSelector` 的 6 种类型须与后端 `rent_escalation_engine` package 中的枚举一致（通过 API 调用，前端不直接引用）。配置器整体超过 250 行时，将 `EscalationPhaseCard` 拆入 `components/EscalationPhaseCard.vue`。
>
> 附：`@file:docs/ARCH.md`（文件复杂度拆分规则）`@file:.github/copilot-instructions.md`

Day 35 · 5月26日（周二）— 各类型参数表单

- `FixedRateForm`（涨幅百分比 + 递增周期年数）
- `FixedAmountForm`（固定金额 ¥/m²/月 + 递增周期）
- `SteppedForm`（`el-table`：年份段 × 单价，可增删行）
- `CpiLinkedForm`（历年 CPI 录入表 + 生效年份选择）
- `EveryNYearsForm`（间隔 N 年 + 涨幅百分比）
- `PostRenovationForm`（免租结束后首年基准价 + 后续叠加规则选择）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现 6 种递增类型参数表单（`FixedRateForm`、`FixedAmountForm`、`SteppedForm`、`CpiLinkedForm`、`EveryNYearsForm`、`PostRenovationForm`）。所有表单均通过 `el-form` ref + `validate()` 校验，**不直接在事件处理函数中提交数据**，由父级 `EscalationPhaseCard` 在"确认"时统一读取。`SteppedForm` 的 `el-table` 行数据用 `ref<SteppedRow[]>([])` 管理，增删行通过 Vue 响应式精细控制。表单字段值的类型必须与 `packages/rent_escalation_engine/lib/models/` 中的对应字段类型一致（number 不用 String 传递）。
>
> 附：`@file:.github/copilot-instructions.md`（架构约束）`@file:docs/ARCH.md`

Day 36 · 5月27日（周三）— 模板管理 + 预测图表

- `EscalationTemplatePage`（保存当前配置为命名模板，按业态分类，支持搜索/编辑/删除/设为默认）
- `RentForecastChart`（admin 端 ECharts 折线图：X 轴年月，Y 轴月租金，全合同期预测，支持年/月切换）

- `ContractFormPage` 集成配置器：保存时将阶段列表序列化为 API JSON 格式

> 💬 **Copilot 提示语**（模板： `/admin-view` ）：
>
> 实现 `EscalationTemplatePage` （模板管理）和 `RentForecastChart` （预测折线图）。**`RentForecastChart` 的数据来自后端 API（`packages/rent_escalation_engine` 计算），组件内不做任何租金数学计算。** 预测期跨越合同全生命周期（可能 5~10 年），数据点可能超过 120 个月，ECharts 需设置 `dataZoom` 并按需聚合为年度数据。模板保存时，阶段配置序列化为 JSON 数组，通过 `useEscalationTemplateStore.saveTemplate()` 提交，不直接调用 HTTP。
>
> 附： `@file:.github/copilot-instructions.md` `@file:docs/backend/data_model.md` （递增规则 JSONB 结构）

Day 37 · 5月28日（周四）— 配置器联调 + 混合分段验证

- 前后端联调：保存递增规则 → 请求 `GET /api/contracts/:id/rent-forecast` → 渲染预测图
- 测试 3 种混合分段场景（如"第1~2年固定 + 第3~4年5%递增 + 第5年CPI挂钩"）与手工计算对比
- 续签对比组件：显示原合同末期租金 vs 新合同起始租金、涨跌幅百分比

> 💬 **Copilot 提示语**（联调用）：
>
> 联调租金递增配置器：配置器保存 → `PUT /api/contracts/:id/escalation-rules` → 触发 `GET /api/contracts/:id/rent-forecast` → `RentForecastChart` 更新。联调过程中若发现前后端数据字段名不一致，修改前端 TypeScript 接口映射来适配后端（**不修改后端枚举/字段命名**）。随后对 3 种混合分段场景：分别用配置器构造、提交、查看预测图，**同时用`packages/rent_escalation_engine` 本地计算同一场景进行对比断言（误差 < 0.01）**，将测试用例写入 `test/rent_escalation_integration_test.dart` 。
>
> 附： `@file:docs/backend/data_model.md` （递增规则 JSONB 格式） `@file:.github/copilot-instructions.md`

Day 38 · 5月29日（周五）— WALE 瀑布图 + M2 集成测试

- `LeaseExpiryWaterfallChart` （admin 端 ECharts 分组柱状图：X 轴年份，Y 轴到期面积 m^2，按业态颜色区分）
- M2 全功能集成测试：合同全状态流转、递增规则联动账单预算、预警生成验证
- RBAC 测试：租务专员可新建合同，不可删除；财务人员不可修改合同

> 💬 **Copilot 提示语**（模板： `/admin-view` ）：
>
> 实现 `LeaseExpiryWaterfallChart` ，X 轴年份（2025~2035），Y 轴 m^2（三业态叠加柱）。颜色严格遵守 CSS 变量 / ECharts 主题色：写字楼→primary，商铺→success，公寓→warning，**不使用固定 hex 颜色**。接着执行 M2 集成测试：①合同 active→expiring_soon→terminated 全状态流转；②递增规则配置后 `GET /api/contracts/:id/rent-forecast` 返回 12 个月预测金额与期望值对比；③RBAC：用财务角色 JWT 调 `DELETE /api/contracts/:id` 期望 403。
>
> 附： `@file:.github/copilot-instructions.md` （RBAC、颜色token） `@file:docs/backend/API_INVENTORY_v1.7.md`

第 9 周（6/1 — 6/5）· M2 收尾

Day 39 · 6月1日（周一）— 商铺营业额分成

- `Contract` 模型增加 `revenueShareRate`（nullable，仅 retail 业态启用）+ `minimumRent`（保底租金）
- 商铺合同表单：选择"保底+分成"后展示分成参数输入区
- `POST /api/contracts/:id/revenue-entries`（录入当月营业额）+ `GET /api/contracts/:id/revenue-preview`（按营业额计算实收租金预览）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 扩展 M2 合同模型支持商铺营业额分成。后端：`Contract` 新增 `revenueShareRate`（`NUMERIC(5,4) nullable`）和 `minimumRent`（`NUMERIC(12,2) nullable`）；迁移 SQL 使用 `ADD COLUMN ... DEFAULT NULL`；实收租金计算逻辑：`max(minimumRent, monthlyRevenue * revenueShareRate)`。**保底分成计算必须放入`packages/rent_escalation_engine`的`revenue_share_calc.dart`（不内联在 Service）**，通过 `path:` 依赖引用。`GET /api/contracts/:id/revenue-preview` 返回 `{"data": {"baseRent": x, "revenueShare": y, "effectiveRent": z}}`。
>
> 附：`@file:docs/backend/data_model.md`（合同模型定义）`@file:.github/copilot-instructions.md`（API 信封格式）

Day 40 · 6月2日（周二）— 合同附件管理完善

- 附件管理组件（前端）：上传 PDF（uni-app `uni.chooseFile` / admin `el-upload`）+ 文件预览/下载（通过 `/api/files/contracts/...` 代理）+ 单个删除确认弹窗
- 后端 RBAC：附件只有合同所属租务专员/管理层可删除，财务只读

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现合同附件管理组件。文件上传通过 API 函数发 `POST /api/contracts/:id/attachments`，进度通过 `onUploadProgress` 回调更新进度条。文件下载/预览地址固定为 `/api/files/contracts/{contractId}/{filename}`（使用 `api_paths.ts` 中的常量，不硬编码路径字符串）。删除附件时先弹确认框（无需二次输入文件名），确认后调用 `DELETE /api/contracts/:id/attachments/:attachmentId`。**上传大小限制通过后端`MAX_UPLOAD_SIZE_MB`环境变量控制，前端不做 MB 硬编码判断。**
>
> 附：`@file:.github/copilot-instructions.md`（文件存储路径规范）`@file:docs/backend/API_INVENTORY_v1.7.md`

Day 41 · 6月3日（周三）— M2 性能优化

- 合同列表 500+ 条分页 SQL `EXPLAIN ANALYZE`：确认 `(unit_id, status)` 复合索引生效
- 前端 `ContractListView` 大列表性能：确认分页加载（非全量渲染）
- 日期重计算优化：`daysUntilExpiry` 在后端 SQL 计算而非前端循环

> 💬 **Copilot 提示语**（模板：`/backend-module`）：

> M2 性能调优。执行 `EXPLAIN ANALYZE SELECT ... FROM contracts WHERE unit_id = $1 AND status = $2 LIMIT 20 OFFSET 0` ，确认 bitmap index scan 生效，扫描行数 < 总量的 10%。若缺少索引，生成迁移 SQL（ `CREATE INDEX CONCURRENTLY` ， `CONCURRENTLY` 不阻断写操作）。 `daysUntilExpiry` 字段改为在 SQL 中计算： `(end_date - CURRENT_DATE) AS days_until_expiry` ，移除前端日期运算循环。**前端合同列表检查：确保使用分页加载（admin 端`el-table`+`el-pagination`，uni-app 端`scroll-view`+触底加载）**；用浏览器 Performance 工具录 5 秒滚动，确认无卡顿。
>
> 附： `@file:docs/ARCH.md` （架构约束：性能要求） `@file:.github/copilot-instructions.md`

Day 42 · 6月4日（周四）— M2 验收对标

- PRD §七核对：WALE 精度（与手工 Excel 误差 < 0.01）、递增规则自动计算、预警 10 分钟内触发
- 更新 `docs/api/m2_contracts.md` （补充联调过程中发现的边界行为）

> 💬 **Copilot 提示语**（模板： `/security-and-test` ）：
>
> M2 验收对标。首先生成 `test/m2_acceptance_test.dart` ，覆盖：①WALE 精度（构造 3 份合同数据，断言 `WaleCalculator.compute()` 结果与手工 Excel 误差 < 0.01）；②递增规则：调用 `rent_escalation_engine` 计算 2 年 FixedRate(5%) 场景，断言月租金序列；③合同状态机： `active` → `submitted_termination` → `terminated` 全路径 HTTP 集成测试。将联调发现的边界行为（如免租期计算、分成合同逾期处理）追加到 `docs/api/m2_contracts.md` "已知行为备注"节。**不得为通过验收而修改验收标准数字**（WALE 精度 0.01 是硬性约束）。
>
> 附： `@file:docs/backend/data_model.md` （WALE计算公式） `@file:.github/copilot-instructions.md`

Day 43 · 6月5日（周五）— M1-M2 跨模块联动

- `UnitDetailPage` 填充当前合同摘要（合同编号 / 月租金 / 到期日 / `daysUntilExpiry` ）
- 楼层热区图颜色源改为 M2 合同到期日驱动（expiringSoon ≤90天 → tertiary 色）
- `TenantDetailPage` 工单 Tab 关联 M4 预留桩（待 M4 完工后回填）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现 M1-M2 跨模块联动。 `useUnitDetailStore` 在加载单元详情时同步调用 `contractApi.getCurrentByUnitId(unitId)` 获取当前合同摘要（**不新建单独接口**，复用 M2 已有接口）；若 `daysUntilExpiry ≤ 90` ， `UnitDetailPage` 的状态色用 warning， `≤ 30` 天用 danger。楼层热区 `FloorMapPage` 的单元颜色逻辑抽取为 `getUnitStatusColor(status, contract)` 纯函数（便于单元测试），**不在组件模板中内联 if-else 颜色判断**。 `TenantDetailPage` 工单 Tab 保留占位并添加注释 `// TODO: Day 52 M4 linkage` 。
>
> 附： `@file:.github/copilot-instructions.md` （UI色彩规范） `@file:docs/ARCH.md` （M1-M2联动设计）
>
> **Milestone 2**：合同状态机全流程跑通；租金递增配置器支持 6 种类型 + 混合分段；WALE 计算误差 < 0.01 年；楼层热区颜色实时联动合同到期日。

Phase 3：M4 物业运营与工单系统

> 时间：2026-06-08（W10）— 2026-06-19（W11），共 10 个工作日

第 10 周（6/8 — 6/12）· M4 后端 + 前端类型/Store

Day 44 · 6月8日（周一）— M4 API Contract + 后端骨架

- 输出 `docs/api/m4_workorders.md`（work*orders* / *suppliers* / *photos* / *cost*entries 端点）
- `work_order.dart` + `supplier.dart`（@freezed，状态枚举 + 优先级枚举），数据迁移 SQL
- M4 路由骨架（Controller 空实现，返回 mock JSON）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 开始 M4 工单模块。先生成 `docs/api/m4_workorders.md`（参照 `docs/backend/API_INVENTORY_v1.7.md` 中 M4 部分，完善请求/响应字段）。后端骨架：创建 `backend/lib/modules/workorders/models/work_order.dart`（freezed，WorkOrderStatus 枚举：submitted/reviewed/processing/pending*acceptance/completed/rejected*；*WorkOrderPriority* 枚举：*normal/urgent/critical*）；生成数据迁移 *SQL*（`*work*orders、suppliers、work*order*photos、cost_entries 四表）；所有 `FK` 使用 UUID REFERENCES ... ON DELETE RESTRICT。`Controller` 骨架返回 {"data": []}` 占位，不实现真实逻辑。
>
> 附：`@file:docs/backend/data_model.md`（工单实体定义）`@file:.github/copilot-instructions.md`（API 信封格式）

Day 45 · 6月9日（周二）— M4 后端 Service + Controller

- `WorkOrderRepository`（CRUD + 按 status / buildingId / assigneeId / priority 多条件查询 + 分页）
- `WorkOrderService`：状态机转换（submitted→reviewed→processing→pending_acceptance→completed / rejected），RBAC 检查（只有审核权限角色可派单）
- `SupplierRepository` + `SupplierService`（CRUD + 按服务类型查询）
- `WorkOrderController`（`GET/POST /api/work-orders`，`PATCH /api/work-orders/:id/status`，`POST /api/work-orders/:id/photos`）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 M4 工单后端核心。`WorkOrderRepository` 的多条件查询使用动态 SQL 拼接（`WHERE TRUE` + 条件 `AND`），所有变量通过参数化传递（`$1, $2...`，绝不字符串插值）。`WorkOrderService.transitionStatus()` 必须显式校验状态合法性（维护一个 `_allowedTransitions` Map，不在代码中散落 if-else），非法转换抛出 `AppException('INVALID_STATUS_TRANSITION', ..., 422)`。状态变更后写入 `audit_logs` 表（操作类型：`work_order_status_change`）。`WorkOrderController` 中每个 handler 不超过 15 行，复杂逻辑下沉 Service，**Controller 只做参数解析 → 调用 Service → 返回响应**。
>
> 附：`@file:.github/copilot-instructions.md`（架构分层规则）`@file:docs/ARCH.md`

Day 46 · 6月10日（周三）— 推送服务后端

- `push_service.dart`：封装 Firebase Cloud Messaging HTTP v1 API（Authorization: Bearer `GoogleServiceAccount` 令牌）
- 工单状态变更触发推送：提单人（进度更新）+ 指派人（新工单通知）
- 桌面/Web 替代：状态变更时写入目标用户的 `alerts` 表（type: work*order*update）；外加 `GET /api/alerts/unread/count` 供 30 秒轮询

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 `push_service.dart`（FCM HTTP v1 推送）。FCM 接入凭证（`Google Service Account JSON` 内容）从 `FCM_SERVICE_ACCOUNT_JSON` 环境变量读取（配置在 `app_config.dart`，缺失时启动失败）。推送逻辑不阻塞主请求：`push_service.sendToUser(userId, title, body)` 内部异步发送，失败仅打印日志不抛异常（推送不影响工单状态机）。**移动端推送失败时必须降级**：在 `alerts` 表写入同一条通知（`type: work_order_update`），确保桌面端轮询 `GET /api/alerts/unread/count` 仍可感知。`app_config.dart` 中 FCM 凭证验证方式：检查 JSON 中 `client_email` 字段存在且非空。
>
> 附：`@file:.github/copilot-instructions.md`（安全：环境变量注入规则）`@file:docs/ARCH.md`

Day 47 · 6月11日（周四）— M4 前端类型/API/Store

- TypeScript 接口：`WorkOrder`，`WorkOrderStatus`，`Supplier`，`CostEntry`（`app/src/types/` + `admin/src/types/`）
- API 函数（`api/modules/workorder.ts`）+ Mock 数据（5 条不同状态工单）
- `useWorkOrderListStore`（状态 Tab 过滤）+ `useWorkOrderDetailStore`（加载详情 + 状态更新）+ `useReportWorkOrderStore`（表单状态管理）
- Store 单元测试覆盖（Vitest）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> M4 前端类型层：`WorkOrder` TypeScript 接口（含 `WorkOrderStatus` + `WorkOrderPriority` 枚举，与后端枚举名严格对应）。API 层：`workorder.ts` 调用 `api_paths.ts` 的路径常量（**不硬编码`/api/work-orders`**）。Store 层：`useWorkOrderListStore` 若超 200 行，将"关键字搜索"分拆为 `useWorkOrderSearchStore`；`useReportWorkOrderStore` 管理报修表单（楼栋/楼层/单元/类型/紧急程度/描述），表单提交成功后通过返回值供页面跳转。测试：每个 Store 覆盖 `initial` / `loading` / `loaded` / `error` / `filter_changed` 五种 case。
>
> 附：`@file:.github/copilot-instructions.md`（领域分层规则）`@file:docs/api/m4_workorders.md`

Day 48 · 6月12日（周五）— M4 UI 工单列表 + 详情

- `WorkOrderListPage`（状态 Tab：待审核 / 处理中 / 待验收 / 已完成）+ `WorkOrderCard`（优先级色标 + 楼栋单元 + 提报时间）
- `WorkOrderDetailPage`：状态时间轴 `Stepper` + 照片网格 `GridView` + 费用明细 `ListTile` + 角色操作按钮（派单/完工/验收/拒绝）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：

> 实现 `WorkOrderListPage` 和 `WorkOrderDetailPage` 。优先级色标：critical→danger，urgent→warning，normal→success（全部通过 CSS 变量 / Element Plus Tag type）。 `WorkOrderDetailPage` 的操作按钮依据当前登录用户角色（从 `useAuthStore().user.role` 读取）动态渲染，**不在组件内硬编码角色字符串**，提取为 `canDispatch(role)` 等工具函数。状态时间轴用 `wd-steps` （uni-app）或 `el-steps` （admin）。照片列表通过 `/api/files/workorders/{id}/{index}.jpg` 代理加载，**不直接暴露存储地址**。
>
> 附： `@file:.github/copilot-instructions.md` （色彩规范、文件存储路径规范） `@file:docs/ARCH.md`

第 11 周（6/15 — 6/19）· M4 前端 报修 + 推送 + 联调

Day 49 · 6月15日（周一）— 移动端报修页

- `ReportWorkOrderPage` ：楼栋选择 → 楼层选择 → 单元选择（三级联动 Dropdown，数据从 M1 API 拉取）
- 问题类型选择（列表：水电 / 空调 / 消防 / 结构 / 其他）+ 紧急程度（一般 / 紧急 / 非常紧急）+ 描述文本框
- `image_picker` / `uni.chooseImage` 照片选择（最多 5 张）+ 预览网格 + 多张并发上传进度条

> 💬 **Copilot 提示语**（模板： `/uniapp-page` ）：
>
> 实现移动端报修页 `ReportWorkOrderPage` 。三级联动（楼栋→楼层→单元）每级选择后触发下级数据加载，用 `useReportWorkOrderStore` 中的 action 管理级联逻辑，**不在组件内直接控制级联状态**。图片上传：最多 5 张，通过 `uni.chooseImage` 选取后存入本地列表，点击 UI 预览网格；提交时并发上传（每张独立调用 `POST /api/work-orders/:id/photos` ），并发上传进度分别更新。紧急程度枚举必须引用 TypeScript 类型定义（ `WorkOrderPriority` ），**禁止组件内出现字符串 "normal/urgent/critical"**。
>
> 附： `@file:.github/copilot-instructions.md` （架构分层） `@file:docs/ARCH.md` （文件存储规范）

Day 50 · 6月16日（周二）— QR 扫码 + 平台差异化

- `QrScanPage` ： `mobile_scanner` 集成，扫码后解析 unitId（二维码内容格式约定： `propos://units/{unitId}` ）→ 自动预填 `ReportWorkOrderPage`
- `PlatformUtils.supportsQrScan` 判断：桌面/Web 端隐藏扫码入口，降级展示手动选择楼栋/楼层/单元的完整表单
- 相机权限请求封装（iOS `permission_handler` ）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` ）：
>
> 实现 `QrScanPage` 和平台差异化入口。uni-app 使用 `uni.scanCode` 扫码，解析格式 `propos://units/{unitId}` （使用 URL 解析 + scheme/host 校验，**不用正则直接匹配字符串**，防止格式变更导致漏解析）；解析成功后 `uni.navigateTo({ url: '/pages/workorders/report?unitId=${unitId}' })` 。admin 端隐藏扫码入口，降级展示手动选择楼栋/楼层/单元的完整表单。
>
> 附： `@file:.github/copilot-instructions.md` （平台差异化） `@file:docs/ARCH.md`

Day 51 · 6月17日（周三）— 费用录入 + 供应商管理

- `CostEntryPage` （完工后录入：材料费 + 人工费 + 供应商关联 + 完工照片上传）

- `SupplierListPage`（供应商列表 + 搜索 + 类型筛选）+ `SupplierFormPage`（新增/编辑）
- 后端：`POST /api/work-orders/:id/cost-entry`，费用写入 `expenses` 表并设 `source: work_order`（M3 NOI 联动接口预留）

> 💬 **Copilot 提示语**（模板：`/backend-module`，接着 `/uniapp-page` + `/admin-view`）：
>
> 后端：`POST /api/work-orders/:id/cost-entry` 校验工单状态必须为 `pending_acceptance` 或 `completed`（其他状态抛 `AppException('INVALID_WORK_ORDER_STATE', ..., 422)`）；写入 `expenses` 表时设 `source = 'work_order'` 和 `source_id = workOrderId`（为 M3 NOI 联动预留，**不修改 M3 expenses 表，只写 source 字段**）。前端 `CostEntryPage`：供应商选择通过搜索弹窗（admin `el-dialog` + 搜索框，uni-app `wd-search` + 列表），**不用下拉菜单内联全部供应商**（避免大量数据渲染）；完工照片上传复用 Day 49 的并发上传逻辑（抽取为 `MultiPhotoUploader` 组件）。
>
> 附：`@file:.github/copilot-instructions.md`（架构约束）`@file:docs/ARCH.md`（M3 NOI 联动预留接口）

Day 52 · 6月18日（周四）— M4 前后端联调

- 完整工单流转联调：移动端报修 → PC 端派单 → 处理中 → 完工费用录入 → 验收完成
- FCM 推送验证（iOS/Android 测试设备，状态变更 → 通知栏出现推送）
- 桌面端轮询验证：工单状态变更 → 10 秒内 `/api/alerts/unread/count` 数值更新

> 💬 **Copilot 提示语**（联调用）：
>
> M4 全链路联调。联调顺序：①移动端 `ReportWorkOrderPage` 提交 → 后端 `work_orders` 表出现新记录；②PC 端 `WorkOrderListView` 刷新列表 → 出现新工单；③派单 `PATCH /api/work-orders/:id/status {status: reviewed, assigneeId: ...}` → FCM 推送到指派人手机；④工单完工录入费用 → `expenses` 表出现 `source: work_order` 记录；⑤验收 `pending_acceptance→completed` → 提单人手机收到推送或桌面端未读数 +1。联调中若发现字段名不一致，只修改前端 TypeScript 接口映射，**不修改后端字段名**（保持 API 约定稳定）。
>
> 附：`@file:docs/api/m4_workorders.md` `@file:.github/copilot-instructions.md`

Day 53 · 6月19日（周五）— M4 验收 + 微信小程序骨架

- 验收：报修全流程 + 成本汇入（Day 51 接口联动确认）+ RBAC（前线员工不可审核/派单）
- 微信小程序骨架：`app.json` 配置 2 页（报修表单页 + 工单状态查询页），API 接口复用后端（不开发推送）

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：
>
> M4 验收 + 微信小程序骨架。RBAC 测试：用 `frontline_staff` 角色 JWT 调 `PATCH /api/work-orders/:id/status {status: reviewed}` → 期望 403；用 `manager` 角色调相同接口 → 期望 200。微信小程序骨架（**仅骨架，不完整实现**）：`app.json` 配置 2 个页面（`pages/report/index` + `pages/status/index`）；`report/index` 表单字段与 uni-app 端一致（楼栋/楼层/单元/类型/描述），`wx.request` 调用同一后端 API（注意小程序不支持 `Authorization: Bearer`，改用 `wx.setStorageSync` 存储 token 并在 header 注入）；**推送能力完全不实现，文档注释标记 [Phase 2]**。小程序代码放入 `/miniprogram/` 目录。

> 附：`@file:.github/copilot-instructions.md`（架构约束：Phase 2 功能不超前实现）`@file:docs/ARCH.md`
>
> **Milestone 3**：工单从提报到完工全链路跑通；移动端 FCM 推送验证成功；维修成本写入 expenses 表接口联调确认。

Phase 4：M3 财务与业财一体化

> 时间：2026-06-22（W12）— 2026-07-17（W15），共 20 个工作日

第 12 周（6/22 — 6/26）· M3 后端 账单 + 收款 + NOI

Day 54 · 6月22日（周一）— M3 API Contract + Invoice 后端

- 输出 `docs/api/m3_finance.md`（invoices / payments / expenses / noi / kpi 端点）
- `invoice.dart` + `payment.dart` + `expense.dart`（@freezed），数据迁移 SQL
- `InvoiceRepository`（CRUD + 按合同/月份/状态查询 + 按维度聚合）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 开始 M3 财务模块。先生成 `docs/api/m3_finance.md`（参照 `docs/backend/API_INVENTORY_v1.7.md` M3 部分，完善 invoices / payments / expenses / noi / kpi 全端点的请求响应字段）。后端 freezed 模型：`Invoice`（`InvoiceStatus` 枚举：pending/partial/paid/overdue/cancelled）、`Payment`（to*account*date、amount、matchedInvoiceId）、`Expense`（category 枚举：maintenance/management/insurance/tax/other；source 枚举：manual/work*order*）。数据迁移 *SQL*：`*invoices*`（含 `*generated*at TIMESTAMPTZ DEFAULT NOW()）、payments、expenses（含 source VARCHAR(20) + source_id UUID nullable`）三张表，所有 TIMESTAMPTZ 列存 UTC。
>
> 附：`@file:docs/backend/data_model.md`（财务实体定义）`@file:.github/copilot-instructions.md`（API 信封格式）

Day 55 · 6月23日（周二）— 账单自动生成（难点）

- `invoice_service.generateMonthlyInvoices(targetMonth)`：遍历所有 `active` 合同，调用 `RentCalculator.compute(phases, targetMonth)` 获取当月租金，生成 `Invoice` + `InvoiceItem`（租金 / 物管费 / 水电代收）
- 免租期判断：当月在免租期内则 `InvoiceItem.isExempt = true`，不计入逾期
- `POST /api/invoices/generate`（触发批量生成，事务包裹，目标 639 条 < 30 秒）+ `POST /api/invoices/export`（Excel 下载）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现账单自动生成 `InvoiceService.generateMonthlyInvoices()`。**整个批量生成必须在单个数据库事务中执行**（PostgreSQL `BEGIN/COMMIT/ROLLBACK`），任意合同生成失败则整批回滚并记录错误。`RentCalculator.compute(phases, targetMonth)` 调用 `packages/rent_escalation_engine`（path 依赖），**不在 Service 内重写递增计算逻辑**。免租期判

> 断：`targetMonth >= freeRentStart && targetMonth <= freeRentEnd`（使用 `DateTime` UTC 比较，不用字符串比较）。性能目标：在 `generateMonthlyInvoices` 末尾记录日志 `"Generated {count} invoices in {ms}ms"`，确保 639 条 < 30000ms；若超时，将逐条 INSERT 改为批量 `INSERT INTO ... VALUES (...),(...),...`。
>
> 附：`@file:docs/backend/data_model.md` `@file:.github/copilot-instructions.md`（无 ORM，原生 SQL）

Day 56 · 6月24日（周三）— 收款核销

- `payment_service.dart`：录入到账信息 → 按合同/金额自动匹配未核销账单 → 标记核销状态，差额处理（多付/少付）
- 逾期账单：调用 `alert_service.createOverdueAlert()` 在第 1/7/15 天生成催收提醒 Alert
- `paymentController`（`POST /api/payments`，`POST /api/invoices/:id/reconcile`）+ 发票号录入（`PATCH /api/invoices/:id/invoice-no`）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现收款核销 `PaymentService`。自动匹配逻辑：按 `contractId` + 金额降序排列未核销账单，贪心匹配（大额优先），差额写入 `payments.unmatched_amount`（多付为正，少付为负）。逾期节点 1/7/15 天使用`business_rules.dart`中的常量（`kOverdueDays = [1, 7, 15]`），禁止硬编码数字。核销操作写 `audit_logs`（`operation: payment_reconciliation`，before/after JSON 含 `invoiceStatus`、`paidAmount`）。`PATCH /api/invoices/:id/invoice-no` 更新发票号时验证：发票号格式正则 `^[A-Z0-9]{8,20}$`（所有验证在后端做，前端不做格式校验替代）。
>
> 附：`@file:.github/copilot-instructions.md`（审计日志要求、常量管理规则）`@file:docs/backend/data_model.md`

Day 57 · 6月25日（周四）— Expense + NOI 计算

- `expense_repository.dart`（CRUD + 按 category / buildingId / month 聚合查询）
- `noi_service.computeNoi(month, {buildingId, propertyType})`：PGI（合同月租合计）- VacancyLoss（空置单元市值估算）+ OtherIncome（停车/储藏室）- OpEx（expenses 聚合）= NOI
- `GET /api/noi/summary`（月度 NOI：全局 + 三业态拆分）+ `GET /api/noi/trend?months=12`

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 NOI 计算服务。`NoiService.computeNoi()` 的四个分量全部通过 SQL 聚合查询获取（PGI、VacancyLoss、OtherIncome、OpEx 各一个 `SELECT` + 聚合，不在 **Dart** 层做 **reduce** 求和）。NOI 计算公式：`NOI = EGI - OpEx = (PGI - VacancyLoss + OtherIncome) - OpEx`，公式注释写在函数上方（与 `copilot-instructions.md` 中一致，不自行发明不同公式）。VacancyLoss 估算 = 空置单元面积 × 市场均价（`market_rent_per_sqm` 字段存于 `buildings` 表，需联 JOIN 查询）。`GET /api/noi/summary` 返回结构：`{"data": {"global": {...}, "office": {...}, "retail": {...}, "apartment": {...}}}`，每个分组含 `pgi, vacancyLoss, otherIncome, opEx, noi` 五个数值字段。
>
> 附：`@file:.github/copilot-instructions.md`（核心计算公式：NOI=EGI-OpEx）`@file:docs/backend/data_model.md`

Day 58 · 6月26日（周五）— 工单成本 → NOI 支出联动

- M4 `CostEntry` 保存时调用 `expense_service.createFromWorkOrder(workOrderId, amount, category)`，自动写 `expenses` 表，`source = 'work_order'`
- `GET /api/expenses/by-category`（按类型汇总：维修 / 物管 / 保险 / 税金 / 其他）
- 回填 Day 51 工单费用录入接口，端到端测试：工单完工 → 费用录入 → NOI 支出更新

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 M4→M3 工单费用联动。`ExpenseService.createFromWorkOrder()` 接收 `workOrderId`、`amount`、`category`：写 `expenses` 表时设 `source = 'work_order'` 和 `source_id = workOrderId`，`category` 固定为 `'maintenance'`（不允许工单来源费用改类别）。端到端测试脚本：① `POST /api/work-orders/:id/cost-entry {materials: 500, labor: 200}` → 响应 200；②立即查 `GET /api/expenses?sourceId=:workOrderId` → 出现总金额 700 的记录；③触发 `GET /api/noi/summary?month=2026-06` → `opEx` 数值增加 700。将测试步骤写入 `docs/api/m3_finance.md` 的"M4联动测试"节。
>
> 附：`@file:docs/backend/data_model.md`（expenses 表结构）`@file:.github/copilot-instructions.md`

第 13 周（6/29 — 7/3）· M3 后端 KPI 引擎

Day 59 · 6月29日（周一）— KPI 指标定义存储

- `kpi_metric_definitions` 表初始化（10 条预定义指标 K01~K10，含 code/name/defaultFullScore/defaultPassScore）
- `kpi_schemes` + `kpi_scheme_metrics`（多对多关联，含权重 + 本方案满分阈值微调）
- `KpiRepository`（方案 CRUD：create / update / delete + 权重校验 sum=1.0）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 初始化 KPI 指标定义体系。`kpi_metric_definitions` 表采用种子数据 SQL（`INSERT ... ON CONFLICT DO NOTHING`，不允许重复插入）初始化 K01~K10 十条记录；`defaultFullScore = 100`，`defaultPassScore = 60` 为常量（写到 `backend/lib/shared/constants/business_rules.dart` 的 `kKpiDefaultFullScore`，不硬编码数字）。`KpiRepository.validateWeights(List<double> weights)` 校验：`weights.reduce(+).toStringAsFixed(4) == '1.0000'`（使用定点精度避免浮点误差）。权重不合法抛 `AppException('INVALID_KPI_WEIGHTS', '权重之和必须为 1.0', 422)`。`kpi_schemes` 与 `kpi_metric_definitions` 多对多关联表 `kpi_scheme_metrics` 含 `weight NUMERIC(5,4)` + `custom_full_score INTEGER nullable`。
>
> 附：`@file:docs/backend/data_model.md`（KPI 实体定义）`@file:.github/copilot-instructions.md`（常量管理规则）

Day 60 · 6月30日（周二）— KPI 数据聚合

- `kpi_service.gatherMetricData(metricCode, schemeId, period)`：按指标 code 从各模块聚合真实数据

- K01 出租率（units 表），K02 收款及时率（invoices/payments），K03 租户集中度（contracts 聚合）
- K04 续约率（contract*status = renewed*），*K05* 工单响应时效（*work*orders 平均时长）
- K06 空置周转天数（units 状态变更历史），K07 NOI 达成率（noi 实际/预算）
- K08 逾期率（invoices 逾期金额占比），K09 递增执行率（递增生效合同数占比），K10 满意度（手动录入）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 `KpiGatherService.gatherMetricData()`。按 `metricCode` switch 分发到不同 SQL 聚合查询（K01~K10 各一个独立私有方法，不在一个巨大 **if-else** 里写全部 **10** 个 **SQL**，每个方法不超过 30 行）。K10（满意度）无自动数据源，读 `kpi_manual_entries` 表（创建该表：`scheme_id, metric_code, period, value NUMERIC(5,2), entered_by UUID`）。K05 工单响应时效 = `AVG(first_response_at - created_at)`，单位秒（PostgreSQL：`EXTRACT(EPOCH FROM (first_response_at - created_at))`），需 `work_orders` 表增加 `first_response_at TIMESTAMPTZ nullable` 列（生成补丁迁移 SQL）。所有聚合结果返回 `double`（`actualValue`），无数据时返回 `null`（由打分服务处理 null→0）。
>
> 附：`@file:docs/backend/data_model.md`（KPI 指标清单）`@file:.github/copilot-instructions.md`

Day 61 · 7月1日（周三）— KPI 打分 + 快照持久化

- 调用 `KpiScorer.score(metric, actualValue)` 计算每个指标得分（0~100）
- $KPI{总分} = \sum(得分i \times 权重_i)$
- `kpi_service.computeAndSaveSnapshot(schemeId, targetPeriod)`：计算所有绑定指标并写入 `kpi_score_snapshots` + `kpi_score_snapshot_items` 表
- `POST /api/kpi/compute`（触发计算 + 持久化，返回快照 ID）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 KPI 打分 + 持久化。打分调用 `packages/kpi_scorer` 的 `KpiScorer.score()`（不在 **Service** 中重写线性插值逻辑）；总分公式 `totalScore = items.map((i) => i.score * i.weight).reduce(+)` 精确到小数点后 2 位（`toStringAsFixed(2)` 到目标精度后再存）。`computeAndSaveSnapshot` 全流程必须在事务中：①计算全部指标→②写 `kpi_score_snapshots`（含 totalScore、period）→③批量写 `kpi_score_snapshot_items`（含 metricCode、actualValue、score、weight），任一失败全部回滚。`POST /api/kpi/compute` 幂等：同一 `schemeId + period` 已有快照时，默认拒绝（`AppException('SNAPSHOT_EXISTS', ..., 409)`），除非请求体带 `"force": true` 才覆盖（先删旧快照再重算）。
>
> 附：`@file:.github/copilot-instructions.md`（核心计算公式：KPI总分）`@file:docs/backend/data_model.md`

Day 62 · 7月2日（周四）— KPI 查询接口

- `GET /api/kpi/schemes`（列表）+ CRUD（创建/更新/删除方案）
- `GET /api/kpi/snapshots?schemeId=&period=`（历史评分快照）+ `GET /api/kpi/snapshots/:id`（含各指标得分明细）

- `GET /api/kpi/ranking?schemeId=&period=` （同方案员工/部门排名）
- `GET /api/kpi/trend?schemeId=&months=6` （某方案历史趋势数组）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 实现 KPI 查询接口集群。分页参数（ `page` + `pageSize` ）必须经过统一校验中间件（ `pageSize` 最大 100，超出时自动 clamp 而非拒绝请求）。 `GET /api/kpi/ranking` 返回 `[{rank, userId, userName, departmentName, totalScore, changeFromLastPeriod}]` （ `changeFromLastPeriod` = 当期总分 - 上期总分，无上期数据时为 `null` ）。 `GET /api/kpi/trend` 返回按 `period` 升序的趋势数组（ `[{period: "2026-06", totalScore: 82.5}]` ，月份粒度）。**删除方案时检查**：若存在历史快照（ `kpi_score_snapshots` ），禁止删除（ `AppException('SCHEME_HAS_SNAPSHOTS', ..., 409)` ），提示用户先归档快照。
>
> 附： `@file:docs/backend/API_INVENTORY_v1.7.md` （KPI 接口定义） `@file:.github/copilot-instructions.md` （分页约定）

Day 63 · 7月3日（周五）— M3 后端验收 + 性能

- 账单批量生成性能：批量生成 639 条， `EXPLAIN ANALYZE` 确认 < 30 秒
- KPI 打分结果与手工计算对比（配置 2 套不同权重方案，结果一致性验证）
- 账单/支出 Excel 导出测试（按业态 / 时间段 / 楼栋维度各导出一次）

> 💬 **Copilot 提示语**（模板： `/security-and-test` ）：
>
> M3 后端验收。生成 `test/m3_backend_test.dart` 覆盖：①调用 `InvoiceService.generateMonthlyInvoices(2026-06)` 计时，断言 < 30000ms；②配置方案 A（K01 权重 0.4、K02 权重 0.6）和方案 B（K01 权重 0.6、K02 权重 0.4），给定相同 actualValue，断言方案 A/B 总分与手工公式 `actualValue_K01*0.4 + actualValue_K02*0.6` 误差 < 0.01；③ `POST /api/kpi/compute` 幂等性：同 schemeId+period 重复调用不带 `force:true` ，期望 409；带 `force:true` 期望 200 且快照被覆盖。Excel 导出：调用 `POST /api/invoices/export?month=2026-06&propertyType=office` ，断言响应 Content-Type 为 `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` 。
>
> 附： `@file:.github/copilot-instructions.md` `@file:docs/backend/data_model.md` （KPI计算公式）

第 14 周（7/6 — 7/10）· M3 前端 账单 + NOI

Day 64 · 7月6日（周一）— M3 前端类型/API/Store

- TypeScript 接口： `Invoice` ， `InvoiceItem` ， `Payment` ， `Expense` ， `NoiSummary` ， `KpiScheme` ， `KpiSnapshot` ， `KpiMetricScore` （ `app/src/types/` + `admin/src/types/` ）
- API 函数（ `api/modules/finance.ts` + `api/modules/kpi.ts` ）+ Mock 数据（各 5 条样本，含已核销/逾期/待核销三种状态）
- Pinia Store： `useInvoiceListStore` ， `usePaymentStore` ， `useExpenseListStore` ， `useNoiDashboardStore` ， `useKpiDashboardStore`
- 全部 Store 单元测试 pass（Vitest）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> M3 前端类型层： `Invoice` TypeScript 接口（含 `InvoiceStatus` 枚举同后端值一一对应； `totalAmount` 从 `items` 计算的 computed，不存冗余字段）； `NoiSummary` （含 `pgi, vacancyLoss, otherIncome, opEx, noi` 五个字段）； `KpiMetricScore` （含 `metricCode, metricName, actualValue, score, weight` ）。Store：若 `useInvoiceListStore` 超 200 行，将"账单核销操作"拆为独立 `useInvoiceReconcileStore` ； `useNoiDashboardStore` 通过 API 模块函数调用（**不直接使用 axios/luch-request**），调用 `getNoiSummary(month)` 和 `getNoiTrend(months: 12)` 。所有 Store 单元测试覆盖 initial/loading/loaded/error， `useKpiDashboardStore` 额外测试 `computeKpi` 触发后 state 更新流。
>
> 附： `@file:.github/copilot-instructions.md` （分层规则） `@file:docs/api/m3_finance.md`

Day 65 · 7月7日（周二）— 账单列表 + 详情 + 核销

- `InvoiceListPage` （Tab 过滤：全部 / 待核销 / 逾期 / 已核销，展示收款进度对比条）
- `InvoiceDetailPage` （费项明细 `el-table` + 核销操作 + 发票号录入 + 状态色标）
- `PaymentFormPage` （录入到账信息：金额 / 到账日期 / 备注）+ 自动匹配账单提示

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现账单管理 UI 三屏。 `InvoiceListPage` 状态色：paid→success（绿），overdue→danger（红），pending→info（灰），partial→warning（橙）（**严格遵守 CSS 变量 / Element Plus type Token**）。收款进度条： `el-progress :percentage="Math.min(paidAmount / totalAmount * 100, 100)"` ， `paidAmount` 已支付总额，超出 100% 时 clamp 防止溢出。 `InvoiceDetailPage` 的核销按钮须做权限保护：通过 `useAuthStore` 检查角色有 `canReconcile` 权限后才用 `v-if` 渲染，**不在后端查数据之前渲染操作按钮**。 `PaymentFormPage` 的到账日期选择器用 `el-date-picker` （中文 locale），选择后更新 Store 的 `toAccountDate` ，格式化为 ISO 8601 传 API。
>
> 附： `@file:.github/copilot-instructions.md` （UI主题色、状态语义） `@file:docs/api/m3_finance.md`

Day 66 · 7月8日（周三）— NOI 实时看板

- `NoiDashboardPage` ：月度三栏卡片（EGI / OpEx / NOI 数值 + 环比变化箭头）
- 三业态 NOI 拆分 `Row` （写字楼 / 商铺 / 公寓各自 NOI 柱状对比）
- 12 月 NOI 趋势折线图（ECharts line）+ 月份横向滑动选择

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现 `NoiDashboardPage` 。顶部三栏数值卡（EGI / OpEx / NOI）数据来自 `useNoiDashboardStore` state（ `v-if="!loading"` 条件渲染，**禁止`v-if="state === 'loaded'"`硬编码状态名**）。环比变化箭头： `changePercent > 0` 显示上箭头图标（CSS 变量 `--color-success` 绿色好）， `< 0` 显示下箭头图标（ `--color-danger` 红色坏）， `== 0` 显示横线图标（ `--color-neutral` 灰色持平）。三业态柱状对比：使用 ECharts 的 bar 图，Y 轴金额格式化为 `¥{金额/万}万` （如 `¥12.3万` ，精确到 0.1 万）。月份选择：横向可滚动月份标签组，选中月份高亮为主色。

> 附：`@file:.github/copilot-instructions.md`（核心计算公式：NOI=EGI-OpEx、UI色彩）`@file:docs/ARCH.md`

Day 67 · 7月9日（周四）— NOI 细项 + 出租率仪表盘

- 出租率环形进度图（ECharts gauge / `el-progress type="circle"`，全局 + 三业态下钻 Tab）
- 空置损失测算展示（空置单元数 × 市值单价 × 面积）
- 本月收款进度 `el-progress`（应收 vs 实收，点击展开未缴款租户列表）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现 NOI 细项仪表盘。出租率环形进度：使用 `el-progress type="circle" :percentage="occupancyRate * 100"`，超过 95% 时颜色用 CSS 变量 `--color-success`（绿色达标），低于 80% 时用 `--color-danger`（红色预警），颜色阈值常量定义在 **`business_rules.ts`（`OCCUPANCY_GOOD_THRESHOLD = 0.95`、`OCCUPANCY_WARN_THRESHOLD = 0.80`），不硬编码数字**。空置损失金额显示：`¥{vacancyLoss}` 来自 `NoiSummary.vacancyLoss`（后端已计算，组件不做面积×单价计算）。收款进度 `el-progress` 点击展开 `el-drawer`，`el-drawer` 内列出未缴款租户（懒加载，分页 `pageSize = DEFAULT_PAGE_SIZE`）。
>
> 附：`@file:.github/copilot-instructions.md`（常量管理、UI主题）`@file:docs/ARCH.md`

Day 68 · 7月10日（周五）— 运营支出录入 UI

- `ExpenseListPage`（按类目 Tab：维修 / 物管 / 保险 / 税金 / 其他；包含工单来源标记）
- `ExpenseFormPage`（手动新增支出：类目 / 金额 / 付款日期 / 楼栋关联 / 备注）

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> 实现 `ExpenseListPage` 和 `ExpenseFormPage`。`ExpenseListPage` 中工单来源费用条目显示 `el-tag`（`source === 'work_order'` 时显示 `<el-tag type="info">工单</el-tag>`）；工单来源条目不可编辑/删除（操作按钮隐藏）。`ExpenseFormPage` 的楼栋选择从资产 API `getBuildings()` 获取（复用 M1 接口，不新建楼栋接口）；类目 `el-select` 的选项使用 `ExpenseCategory` 枚举遍历渲染，枚举显示名称通过 `EXPENSE_CATEGORY_LABEL` 映射对象实现，不在组件内 switch 枚举转中文字符串。
>
> 附：`@file:.github/copilot-instructions.md`（UI主题色）`@file:docs/ARCH.md`（M3-M4联动设计）

第 15 周（7/13 — 7/17）· M3 前端 KPI 仪表盘 + 联调

Day 69 · 7月13日（周一）— KPI 方案配置器

- `KpiSchemePage`（方案列表 + 新建入口 + 当前生效标记）
- `KpiSchemeFormPage`：
  - 指标勾选 `el-checkbox`（10 个预定义指标，含说明文字）
  - 权重输入 `el-input-number`（实时合计校验 = 100% 提示）
  - 满分/及格阈值微调 `el-slider`
  - 评估周期、适用对象（部门/员工）配置

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：

> 实现 KPI 方案配置器。 `KpiSchemeFormPage` 权重合计实时校验：在 `useKpiSchemeFormStore` 中维护 `weights: Record<string, number>` ref，每次输入变更计算 `totalWeight = Object.values(weights).reduce((a, b) => a + b, 0)` ；若 `Math.abs(totalWeight - 1.0) > 0.0001` 则在页面顶部显示 `el-alert` （ 权重合计 `${(totalWeight*100).toFixed(1)}%`，请调整至 `100%` ），保存按钮禁用（不提交不合法权重）。 `el-slider` 组件满分阈值范围 50~100；及格阈值不能高于满分阈值（ `:max="customFullScore"` ，响应式联动）。方案表单超 150 行时拆出 `components/KpiMetricWeightItem.vue` 和 `components/KpiThresholdSlider.vue` 。
>
> 附： `@file:.github/copilot-instructions.md` （文件复杂度拆分规则） `@file:docs/ARCH.md`

Day 70 · 7月14日（周二）— KPI 评分看板

- `KpiDashboardPage` ：当期方案总览卡（总分 + 等级标签）+ 各指标得分雷达图（ECharts radar）
- 员工/部门排名列表（头像 + 姓名 + 得分 + 名次变化箭头）
- 指标下钻：点击雷达图某顶点 → `el-drawer` 展示原始数据明细

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现 `KpiDashboardPage` 。等级标签： `totalScore >= 90` → 等级"优秀"（ `el-tag type="success"` ）， `>= 75` → "良好"（ `type="primary"` ）， `>= 60` → "合格"（ `type="warning"` ）， `< 60` → "不合格"（ `type="danger"` ）；**等级阈值存入 `business_rules.ts`（`KPI_EXCELLENT_SCORE = 90` 等），不硬编码**。雷达图用 ECharts 的 radar 图，每个顶点对应一个 KPI 指标， `data` 用当期指标得分（0~100）；点击顶点事件通过 ECharts 的 `click` 回调获取指标索引，映射到 `KpiMetricScore` 后打开 `el-drawer` 。排名箭头： `changeFromLastPeriod > 0` → 上箭头（success 色）， `< 0` → 下箭头（danger 色）， `null` 或 `0` → 横线（neutral 色）。
>
> 附： `@file:.github/copilot-instructions.md` （UI色彩 token、常量管理） `@file:docs/ARCH.md`

Day 71 · 7月15日（周三）— KPI 历史趋势 + 导出

- 历史折线图（过去 6~12 月 KPI 总分曲线 + 各指标分项趋势可切换）
- 同比/环比数值对比显示（与上月、去年同期）
- `KpiExportButton` （调用 `POST /api/kpi/snapshots/:id/export` 下载 PDF 报告，浏览器下载）

> 💬 **Copilot 提示语**（模板： `/uniapp-page` + `/admin-view` ）：
>
> 实现 KPI 历史趋势和导出。折线图用 ECharts 的 line 图，支持"总分"和"各指标分项"两种视图切换（ `el-radio-group` ）；分项视图时叠加多条折线（每条颜色来自 CSS 变量的不同 token，不使用固定颜色数组）。同比/环比： `useKpiDashboardStore` 提供 `monthOverMonth` （与上月差值）和 `yearOverYear` （与去年同期快照差值），快照不存在时显示 `'N/A'` 。 `KpiExportButton` 点击流程：调用 API → 收到 `Blob` 响应 → `URL.createObjectURL(blob)` + `<a>` 标签触发下载（ `kpi_{period}.pdf` ），不弹文件保存对话框。
>
> 附： `@file:.github/copilot-instructions.md` （UI主题色） `@file:docs/api/m3_finance.md` （KPI导出接口）

Day 72 · 7月16日（周四）— M3 前后端联调

- 账单自动生成联调：M2 合同递增规则 → `POST /api/invoices/generate` → 前端列表即时刷新
- KPI 联调：配置方案 → `POST /api/kpi/compute` → 前端雷达图更新
- NOI 联调：工单费用录入 → expenses 更新 → NOI 看板数值变化

> 💬 **Copilot 提示语**（模板：`/uniapp-page` + `/admin-view`）：
>
> M3 全链路联调。联调清单（按顺序执行，每步截图记录结果）：① `POST /api/invoices/generate {month: "2026-06"}` → 前端 `InvoiceListPage` 刷新（Store 在操作完成后主动调用 `fetchInvoices()`）；②配置 KPI 方案 A → `POST /api/kpi/compute {schemeId, period}` → `KpiDashboardPage` 雷达图数值更新；③工单录入费用 1000 元 → `GET /api/noi/summary?month=2026-06` opEx +1000 → 前端 NOI 看板减少 1000。联调发现字段不一致时，**统一修改前端 TypeScript 接口映射，记录到`docs/api/m3_finance.md` 的"联调备注"节**（不同步修改后端）。
>
> 附：`@file:docs/api/m3_finance.md` `@file:.github/copilot-instructions.md`

Day 73 · 7月17日（周五）— M3 验收对标

- PRD §七逐项：NOI 实时展示一致性、账单 < 30 秒、KPI 2 套方案打分与手工一致
- 权限验证：财务人员可录入支出/核销账单，不可配置 KPI 方案（403）
- 记录 M3 → M5 穿透视角联动待办（NOI 看板增加穿透口径切换）

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：
>
> M3 验收对标。生成 `docs/m3_acceptance_checklist.md`，逐项记录：①NOI 数值对标（前端展示值 vs 数据库查询值，误差 < 0.01 元）；②账单生成 639 条计时（记录实际毫秒数）；③KPI 方案 A/B 打分与手工计算一致（记录手工公式和实际得分）。RBAC 安全测试：用 `finance_staff` 角色 JWT 调 `POST /api/kpi/schemes` → 期望 403；用 `manager` 角色调相同接口 → 期望 200。M3→M5 穿透联动待办：在 `useNoiDashboardStore` 内添加注释 `// TODO: Day 82 M5 penetration mode - add subLandlord scope toggle for occupancy calculation`，不现在实现。
>
> 附：`@file:.github/copilot-instructions.md`（RBAC约定） `@file:docs/PRD.md`（PRD §七验收标准）
>
> **Milestone 4**：账单自动生成完成（三业态多费项）；NOI 三业态拆分正确；KPI 两套方案自动打分与手工一致；工单费用 → NOI 支出链路打通。

Phase 5：M5 二房东租赁信息穿透管理

> 时间：2026-07-20（W16）— 2026-07-31（W17），共 10 个工作日

第 16 周（7/20 — 7/24）· M5 后端 + 外部门户

Day 74 · 7月20日（周一）— M5 API Contract + 行级隔离后端

- 输出 `docs/api/m5_subleases.md`（subleases 端点 + 外部门户专用端点）

- `sublease.dart`（@freezed，`idNumber` 字段标注 `// encrypted`，API 响应脱敏）
- `SubleaseRepository`：所有查询强制附加 `WHERE master_contract_id = ANY($subLandlordScope)` 行级隔离，绝不通过应用层过滤代替 SQL 过滤

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 开始 M5 二房东模块，**安全是本模块核心**。`SubleaseRepository` 中所有查询必须在 SQL WHERE 子句加行级隔离条件（`WHERE master_contract_id = ANY($1::uuid[])`，`$1` 为 JWT 中解析出的 `subLandlordScope`），**绝对禁止在 Dart 代码层过滤替代 SQL 层隔离**。`sublease.dart` freezed 模型：`idNumber` 字段添加注释 `// encrypted: AES-256`；所有涉及 `idNumber` 的接口响应，通过 `MaskingUtils.maskIdNumber(raw)` 处理（只保留后 4 位，其余替换为 `*`）。生成 `docs/api/m5_subleases.md`（内外两套端点分开文档：内部管理端 `/api/subleases`、外部门户端 `/api/portal/subleases`，后者需额外校验 `sub_landlord` 角色）。
>
> 附：`@file:.github/copilot-instructions.md`（架构约束：行级隔离必须在 SQL 层）`@file:docs/backend/data_model.md`（子租赁实体）

Day 75 · 7月21日（周二）— M5 Service + 审核流

- `SubleaseService`：子租赁 CRUD + 审核状态机（pending_review→approved / rejected），拒绝时必填理由
- 审核通过/拒绝均记录 `audit_logs`（before/after JSON 完整对比）
- `sublease_import_service.dart`：解析二房东 Excel 模板，校验（单元必须在主合同覆盖范围、子租赁到期日 ≤ 主合同到期日、同单元无重叠在租），批量 INSERT

> 💬 **Copilot 提示语**（模板：`/backend-module`）：
>
> 实现 M5 核心业务逻辑。`SubleaseService.review(subleaseId, {action, rejectionReason})` 审核时须验证：① `action` 只允许 `'approved'` 或 `'rejected'`；② `rejected` 时 `rejectionReason` 必填（否则 `AppException('REJECTION_REASON_REQUIRED', ..., 422)`）；③审核操作写 `audit_logs`（`operation: 'sublease_review'`，`before: {status: 'pending_review'}`，`after: {status: action, rejectionReason}`）。`SubleaseImportService.importFromExcel()` 校验逻辑：每行独立校验，失败行记入错误列表，**成功行和失败行分别返回**（分批 INSERT，不因一行失败回滚全批），最后返回 `{successCount, failedRows: [{rowIndex, reason}]}`。
>
> 附：`@file:.github/copilot-instructions.md`（审计日志要求）`@file:docs/backend/data_model.md`

Day 76 · 7月22日（周三）— M5 Controller + 提醒接口

- `SubleaseController`（内部管理端：CRUD + 审核操作 `PATCH /api/subleases/:id/review`）
- `POST /api/subleases/external`（外部门户专用，要求角色 `sub_landlord`，仅限 JWT 中 `subLandlordScope` 范围）
- `POST /api/scheduler/run-sublease-reminders`（遍历所有二房东合同，发送月度填报提醒邮件/短信）

> 💬 **Copilot 提示语**（模板：`/backend-module`）：

> 实现 M5 Controller 层。 `SubleaseController` 区分两套路由：内部管理 `/api/subleases` （需 `is_internal_admin` 或 `leasing_manager` 角色）、外部门户 `/api/portal/subleases` （需 `sub_landlord` 角色，额外中间件验证`subLandlordScope` 非空）。 `POST /api/portal/subleases` 中间件流水线： `authMiddleware → roleMiddleware('sub_landlord') → subLandlordScopeMiddleware → controller` （ `subLandlordScopeMiddleware` 解析 JWT 中 `subLandlordScope` 字段并注入 `context.locals` ，若为空则 403 拒绝）。 `run-sublease-reminders` 接口要求 `scheduler` 角色（内部定时任务专用角色，不对外暴露），实际发送渠道（邮件/短信）通过 `NotificationService` 抽象，Phase 1 仅写日志（实现为 console 输出）。
>
> 附： `@file:.github/copilot-instructions.md` （RBAC、架构分层） `@file:docs/backend/API_INVENTORY_v1.7.md`

Day 77 · 7月23日（周四）— 外部二房东填报门户（admin 端独立路由）

- 独立路由 `/sublease-portal` （admin 端同代码库，Vue Router `beforeEach` 守卫：sub_landlord 角色自动重定向此路由）
- `SubleasePortalLoginPage` （独立登录页，Logo/文案与主管理后台区分）
- `SubleasePortalHomePage` ：显示自身主合同覆盖单元列表，空置/已租/待审核状态标记
- `SubleaseFormPage` （逐条填报：租客信息 / 租金 / 入住状态，含字段实时校验）

> 💬 **Copilot 提示语**（模板： `/admin-view` ）：
>
> 实现二房东外部填报门户（admin 端独立路由，不是独立项目，共享同一代码库）。Vue Router `beforeEach` 守卫：登录后若 `user.role === 'sub_landlord'` 自动 `next('/sublease-portal')` ，禁止 **sub_landlord 角色访问`/buildings`、`/contracts`** 等内部路由（在 `router/index.ts` 的全局守卫中统一处理，不在每个页面组件内做角色判断）。 `SubleasePortalLoginPage` 复用 `useAuthStore` （不实现独立登录逻辑）；UI 仅替换 Logo 图片和页面标题文字。 `SubleaseFormPage` 的证件号（ `idNumber` ）字段必须 `type="password"` （密码模式），提交时发 `/api/portal/subleases` 接口。
>
> 附： `@file:.github/copilot-instructions.md` （架构约束：行级隔离） `@file:docs/ARCH.md` （路由设计）

Day 78 · 7月24日（周五）— 外部门户 Excel 上传 + 提交确认

- 模板下载按钮（ `GET /api/subleases/template` ，返回预填表头的 Excel 文件）
- Excel 批量上传 + 后端返回校验结果（成功行数 + 失败行明细），前端 `el-table` 展示错误行
- 提交后：状态变为"待审核"，不可再次编辑（只读显示），展示提交时间戳
- 变更历史 Tab：按时间排序的历史提交记录（修改前后字段对比）

> 💬 **Copilot 提示语**（模板： `/admin-view` ）：
>
> 实现外部门户 Excel 上传流程。前端： `el-upload` 选择 `.xlsx` 文件（ `accept=".xlsx"` ）→ `FormData` 上传到 `POST /api/portal/subleases/import` ；后端返回 `{data: {successCount: N, failedRows: [{rowIndex, reason}]}}` ，前端用 `el-table` 展示失败行（行号 + 错误原因）。提交确认按钮：先显示 `ElMessageBox.confirm` （"共 N 条记录将提交审核，提交后不可修改，确认？"），确认后调 `POST /api/portal/subleases/submit` ，提交成功后整个表单变为只读（ `useSubleasePortalStore` 的 `isSubmitted = true` ，所有 `el-input` 改为 `disabled` ）。历

> 史记录 Tab：按 `submittedAt DESC` 排列，每次提交显示 `el-tag`（已审核/待审核/已拒绝），点击展开差异对比（简单 `Diff` 格式：前值 → 后值）。
>
> 附：`@file:.github/copilot-instructions.md`（安全：文件上传限制） `@file:docs/api/m5_subleases.md`

第 17 周（7/27 — 7/31）· M5 前端 内部管理 + 穿透看板

Day 79 · 7月27日（周一）— M5 前端类型/API/Store

- TypeScript 接口：`Sublease`，`SubleaseStatus`，`SublandlordOverview`，`SubleaseReviewRecord`（`admin/src/types/`）
- API 函数（`api/modules/sublease.ts`）+ Mock 数据
- `useSubleaseListStore`（按主合同/状态过滤）+ `useSubleaseReviewStore`（加载审核相关详情）

> 💬 **Copilot 提示语**（模板：`/admin-view`）：
>
> M5 前端类型层。`Sublease` TypeScript 接口：`idNumber` 字段类型 `string`，添加文档注释 `/** Already masked: only last 4 digits. Never store raw value on client. */`。`SublandlordOverview` 含 `masterContractRent`（主合同租金）和 `terminalTotalRent`（终端租金合计，用于溢价计算），这两个字段由后端计算返回，**组件不做除法计算溢价率**。Store：`useSubleaseListStore` 过滤维度有"全部/待审核/已审核/已拒绝"；`useSubleaseReviewStore` 加载详情时同步加载审核历史（`loadDetail()` 调用两个 API 函数）。测试：`useSubleaseReviewStore` 测试审核"同意"和"拒绝缺少 reason"两种场景，后者期望 `error.value` 为 `'rejection_reason_required'`。
>
> 附：`@file:.github/copilot-instructions.md`（证件号安全要求） `@file:docs/api/m5_subleases.md`

Day 80 · 7月28日（周二）— M5 内部管理 UI

- `SubleaseManagementPage`：按主合同分组 `el-collapse`，子租赁列表（单元号 / 终端租客 / 租金 / 状态）
- 待审核条目高亮（使用 CSS 变量 `--color-warning` 背景）+ 快速审批操作按钮
- `SubleaseDetailPage`（查看子租赁详情 + 审核历史 + 证件脱敏展示）
- `SubleaseFormPage`（内部录入：单元下拉限主合同范围，起止日期校验）

> 💬 **Copilot 提示语**（模板：`/admin-view`）：
>
> 实现 M5 内部管理 UI。`SubleaseManagementPage` 按主合同分组：`el-collapse-item` 的标题展示主合同编号 + 二房东名称，内容懒加载该主合同下的子租赁列表（**不一次性加载全部**，点击展开时触发 `useSubleaseListStore.fetchByMasterContract(contractId)`）。待审核条目：行背景高亮（CSS 变量 `--color-warning` 浅色背景）+ 行尾 `el-button type="primary" size="small"` "审核"按钮，点击跳转 `SubleaseDetailPage`（附带 subleaseId 参数）。`SubleaseDetailPage` 中证件号字段显示时**使用 model 中已脱敏的 `idNumber` 字段**，不做任何额外处理（后端已保证脱敏）；快速审核弹窗（`ElMessageBox.prompt`）中拒绝时必须填写理由，**不允许空理由提交**。
>
> 附：`@file:.github/copilot-instructions.md`（UI色彩、证件号安全） `@file:docs/ARCH.md`

Day 81 · 7月29日（周三）— M5 穿透分析看板

- `SublandlordDashboardPage`：
  - 每家二房东总览卡（主合同租金 / 已填报数 / 终端出租面积 / 终端空置面积）
  - 转租溢价分析：终端均价 vs 主合同单价 + 溢价率百分比（ColoredBox 红绿标记）
  - 穿透出租率 vs 整体出租率双条进度条对比
  - 子租赁集中到期预警时间轴
  - 填报完整度监控（已填报单元数 / 主合同覆盖总单元数 + 进度环）

> 💬 **Copilot 提示语**（模板：`/admin-view`）：
>
> 实现 `SublandlordDashboardPage` 穿透分析看板。溢价率 = `(terminalAvgRent - masterContractUnitRent) / masterContractUnitRent * 100%` **（使用`SublandlordOverview`中后端已计算的`premiumRate`字段，组件中无除法计算）**；溢价率 > 0 时填色 CSS 变量 `--color-success` 浅色背景（绿色，转租增值），< 0 时填 `--color-danger` 浅色背景（红色，亏损）。双条进度条：纵向排列两个 `el-progress`，一个 `:percentage="penetrationOccupancyRate * 100"`（穿透出租率），一个 `:percentage="overallOccupancyRate * 100"`（整体出租率），显示各自百分比标签。填报完整度进度环：`el-progress type="circle" :percentage="totalUnits ? reportedCount / totalUnits * 100 : 0"`，`totalUnits === 0` 时不渲染（防 NaN）。
>
> 附：`@file:.github/copilot-instructions.md`（UI色彩 token）`@file:docs/backend/data_model.md`（子租赁穿透视角）

Day 82 · 7月30日（周四）— 楼层热区穿透模式 + 跨模块联动

- `FloorMapPage` 增加"穿透模式" `Switch`（`AppBar` 操作区）
- 穿透模式开启时：单元 Tooltip 显示终端租客名称 / 实际月租金 / 入住状态（从 sublease API 聚合）
- `ContractDetailPage` 增加"子租赁"Tab（展示该主合同下所有子租赁记录列表）
- M3 `NoiDashboardPage` 增加"穿透视角"切换（按终端口径统计出租率，区别于主合同口径）

> 💬 **Copilot 提示语**（模板：`/admin-view`）：
>
> 实现楼层热区穿透模式联动。`useFloorMapStore` 增加 `isPenetrationMode` ref；顶栏 `el-switch` 切换时更新该状态。穿透模式下：单元热区组件加载 sublease API `getByUnitId(unitId)` 数据（**懒加载，仅点击展开 Tooltip 时发请求**，防止热区初始化 639 个并发请求）；`ContractDetailPage` 新增"子租赁" Tab（`el-tabs` 增加一个 TabPane，子租赁列表由 `useSubleaseListStore` 提供数据）。M3 `NoiDashboardPage` 穿透视角切换：`el-radio-group`（"主合同口径" / "终端口径"）选中后更新 `useNoiDashboardStore.viewMode`，重新拉取对应 API（`GET /api/noi/summary?mode=penetration`）；**总计算公式不变，只是数据来源切换。**
>
> 附：`@file:.github/copilot-instructions.md`（架构约束、UI色彩）`@file:docs/ARCH.md`（M5 穿透设计）

Day 83 · 7月31日（周五）— M5 联调 + 验收 + 安全测试

- 外部门户全流程联调：二房东登录 → 查看单元 → 填报 → Excel 上传 → 提交待审核 → 内部审核通过 → 数据生效

- 行级隔离安全测试：使用二房东 A 的 JWT 发送请求，强制携带二房东 B 的 `master_contract_id` ，后端必须返回 403 / 空结果
- 审计日志完整性：每次提交/修改/审核均有对应 `audit_logs` 记录（before/after 内容非 null）
- RBAC：二房东角色不可访问 `/api/units` 、 `/api/contracts` 等内部端点（403 测试）

> 💬 **Copilot 提示语**（模板： `/security-and-test` ）：
>
> M5 安全验收（**本日测试结果直接决定上线 Gate**）。必须执行并记录：①行级隔离穿透测试：获取二房东 A 的有效 JWT，在请求体中指定二房东 B 的 `masterContractId` ，调 `GET /api/portal/subleases?masterContractId={B_id}` 期望返回空数组（不是 403，而是空结果，**因为 SQL WHERE 过滤无结果**）；再调 `POST /api/portal/subleases {masterContractId: B_id}` 期望 403（业务层再次验证 scope）。②RBAC 越权测试：用 `sub_landlord` JWT 调 `GET /api/units` → 期望 403；调 `GET /api/contracts` → 期望 403。③审计日志验证：审核通过后立即查 `audit_logs WHERE operation = 'sublease_review'` ，断言 `before` 含 `status: 'pending_review'` ， `after` 含 `status: 'approved'` ，before/after 均非 null。将测试结果截图存入 `docs/security/m5_security_test_report.md` 。
>
> 附： `@file:.github/copilot-instructions.md` （行级隔离、审计日志、RBAC约束） `@file:docs/ARCH.md`
>
> **Milestone 5**：外部填报门户可独立访问，行级隔离安全验证通过；子租赁审核流完整；穿透模式在楼层热区正确叠加终端租客信息；二房东角色数据不出圈。

Phase 6：集成测试 + 数据初始化

> 时间：2026-08-03（W18）— 2026-08-14（W19），共 10 个工作日

第 18 周（8/3 — 8/7）· 全模块集成 + 数据初始化

Day 84 · 8月3日（周一）— 全模块跨领域联动验证

- M1 单元状态 ← M2 合同到期日驱动（热区颜色实时一致性，模拟修改合同 endDate 验证）
- M4 工单费用 → M3 NOI 支出（录入工单费用 → NOI 支出金额变化断言）
- M3 KPI 跨模块数据抽取：修改 M1/M2/M4 数据后重触发 KPI 计算，结果变化方向符合预期
- M5 子租赁审核通过 → M3 穿透视角出租率更新

> 💬 **Copilot 提示语**（模板： `/security-and-test` ）：
>
> Phase 6 全模块跨领域联动验证测试。生成 `test/integration/cross_module_integration_test.dart` ，覆盖四条联动链路：①修改合同 `endDate = now + 25d` → 刷新 `GET /api/units/:id` → 断言单元状态为 `expiring_soon` （热区颜色源确认）；②录入工单费用 1500 → `GET /api/noi/summary?month={current}` → 断言 `opEx` 增加 1500；③触发 `POST /api/kpi/compute` 后调 `GET /api/kpi/snapshots/:id` → 断言 K01 出租率指标 `actualValue` 与 `GET /api/noi/summary` 的出租率数值一致（数据来源统一）；④子租赁审核通过后调 `GET /api/noi/`

> `summary?mode=penetration` → 断言穿透出租率变化。**测试结果必须自动化断言（不依赖人工观察），全部 pass 方可继续 Day 85。**
>
> 附：`@file:.github/copilot-instructions.md` `@file:docs/ARCH.md`（跨模块联动设计）

Day 85 · 8月4日（周二）— 预警全链路 + 推送端到端

- 合同到期预警全链路：手动修改合同 endDate 为 20 天后 → `POST /api/scheduler/run-alerts` → 10 分钟内 `GET /api/alerts` 出现新预警
- 逾期账单预警：手动将账单 dueDate 设为 1/7/15 天前 → 触发 → 验证 Alert 类型和接收人
- FCM 推送端到端：工单状态变更 → iOS/Android 测试机收到通知（截图记录）
- 桌面端轮询：工单状态变更 → PC 桌面端 30 秒内角标更新

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：
>
> 预警全链路端到端测试。分别测试三类预警：①合同到期预警：`UPDATE contracts SET end_date = NOW() + INTERVAL '20 days' WHERE id = :id`，调 `POST /api/scheduler/run-alerts`，立即调 `GET /api/alerts?type=contract_expiry` 断言出现该合同的预警（`daysUntilExpiry` 在 `kAlertDays`（90/60/30）节点触发的断言分别写）；②逾期账单三节点：分别设 due*date* 为 *1/7/15* 天前，各触发一次，断言 `*GET /api/alerts?type=payment*overdue` 出现对应节点的提醒；③桌面轮询：检查 GET /api/alerts/unread/count 在工单状态变更 30 秒内确实递增（可用 `shell` 脚本循环轮询断言）。将三类测试命令整理到 docs/ops/alert*test*playbook.md`（运维操作手册）。
>
> 附：`@file:.github/copilot-instructions.md`（预警常量定义在 business_rules.dart） `@file:docs/ARCH.md`

Day 86 · 8月5日（周三）— 数据初始化 Excel 模板制作

- 制作三份单元导入模板（写字楼 / 商铺 / 公寓），包含：字段列头、类型说明行（注释）、数据验证规则（下拉枚举）、3 条样本行
- 制作合同历史导入模板（含递增规则 JSONB 序列化格式说明）
- 制作子租赁信息导入模板（供二房东使用，含字段校验说明）
- 制作操作手册 v1.0（面向运营团队的导入操作步骤说明，含截图）

> 💬 **Copilot 提示语**：
>
> 制作数据初始化 Excel 模板和操作手册。Excel 模板通过 Dart 脚本（`scripts/gen_import_templates.dart`）生成（使用 `excel` package，不手写 xlsx），字段顺序和枚举值与数据库约束一致（**枚举下拉选项必须与`PropertyType`、`UnitStatus`、`ContractStatus` 等枚举值完全一致**，不自造值）。合同模板中递增规则列说明：格式为 JSON 字符串（提供 2 个样本：`{"type":"fixed_rate","rate":0.05}` 和 `{"type":"stepped","steps":[...]}`），备注列写明可参考 `docs/api/m2_contracts.md`。操作手册 `docs/ops/data_import_guide.md`：分步截图 + 常见错误处理（列举至少 5 个常见报错码及解决方法）。
>
> 附：`@file:docs/backend/data_model.md`（实体字段定义） `@file:.github/copilot-instructions.md`（业务规则常量）

Day 87 · 8月6日（周四）— 实际数据导入（配合运营团队）

- 写字楼单元数据批量导入（约 441 套），验证导入结果（楼层 / 单元号 / 面积核对）
- 商铺 + 公寓批量导入（约 198 套）
- 实际 CAD 文件处理：.dwg → SVG 批量转换，在前端楼层平面图真实渲染验证
- 历史在租合同导入（预估 500+ 条，分批次）

> 💬 **Copilot 提示语：**
>
> 配合运营团队执行实际数据导入。导入脚本 `scripts/import_units.dart` 需支持"预演模式"（ `--dry-run` 标志：校验所有数据但不写库，输出校验报告），**生产导入前必须先跑 dry-run**。导入时每批 50 条，每批完成后打印进度（ `已导入 N / 总计 M 条` ），出错则停止当批并输出错误列表（**不中断剩余批次**）。CAD 转 SVG：对每栋楼拿到实际 .dwg 文件后，用已有 `scripts/convert_cad.dart` 处理，输出到 `floors/{building_id}/{floor_id}.svg` ；转换完成后在前端打开对应楼层的 `FloorMapPage` ，**目测验证单元轮廓与实际平面图一致**（如不一致记录不一致楼层编号，转入 Phase 2 处理）。合同导入：分批每次 100 条，验证 `contracts` 表 `escalation_rules` 字段 JSONB 格式正确（不为 null 的记录随机抽 10 条用 `packages/rent_escalation_engine` 解析验证）。
>
> 附： `@file:.github/copilot-instructions.md` （文件存储路径规范：floors/{building*id}/{floor*id}.svg） `@file:docs/ARCH.md`

Day 88 · 8月7日（周五）— 财务 + KPI 初始数据

- 录入当前未结账单（财务人员协作），标记历史付款记录为已核销
- 初始化 KPI 方案：租务部（K01/K02/K04/K09 四指标）+ 财务部（K02/K07/K08 三指标）+ 物业运营部（K05/K06 两指标 + K10）
- 补录 6 个月 KPI 历史快照（为趋势图提供初始数据，可用估算值）
- 二房东账号创建（各二房东账号 + 绑定主合同范围）

> 💬 **Copilot 提示语**（模板： `/backend-module` ）：
>
> 初始化财务 + KPI 数据。生成种子数据脚本 `scripts/seed_finance.dart` （不修改生产代码）：①初始化三个 KPI 方案（租务部/财务部/物业运营部），权重经验值写死在脚本中，脚本运行前校验总权重 = 1.0；②补录 6 个月历史 KPI 快照（ `kpi_score_snapshots` ），K10 满意度用估算值（85），其余指标调用 `KpiGatherService.gatherMetricData()` 从实际导入数据取真实值；③二房东账号：通过现有 `POST /api/users` （管理员端）创建，角色 `sub_landlord` ， `subLandlordScope` 在 `user_permissions` 表中关联对应 `masterContractId` 数组。运行种子脚本前先备份数据库（ `pg_dump` ）。
>
> 附： `@file:.github/copilot-instructions.md` （安全：证件号加密存储） `@file:docs/backend/data_model.md` （用户权限模型）

第 19 周（8/10 — 8/14）· 全量回归 + 性能 + 安全

Day 89 · 8月10日（周一）— 全量功能回归测试

- 按 PRD §七验收标准逐项测试，制作测试矩阵（功能点 / 测试步骤 / 预期结果 / 实际结果 / Pass or Fail）

- 重点验收项：WALE 精度（< 0.01 年）/ 递增规则计算准确性 / KPI 打分与手工一致 / 账单生成 < 30 秒 / 穿透看板数据

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：
>
> 生成全量回归测试矩阵文档 `docs/qa/regression_test_matrix.md`。格式模板（Markdown 表格）：`| 测试 ID | 模块 | 功能点 | 测试步骤 | 预期结果 | 实际结果 | 状态 |`，初始生成 50+ 条测试用例覆盖 M1~M5 全功能点。**重点标注的硬性验收项**（加粗标注，Fail 则阻断上线）：①WALE 精度 < 0.01 年；②账单批量生成 < 30 秒；③KPI 打分与手工一致（< 0.01）；④行级隔离安全（跨 scope 请求返回空/403）；⑤证件号 API 响应脱敏（全局 grep `SELECT.*id_number` 确认后端返回已脱敏值）。逐项执行后在"实际结果"列填写真实值，"状态"列填 ✅ `Pass` 或 ❌ `Fail(原因)`。
>
> 附：`@file:docs/PRD.md`（PRD §七验收标准）`@file:.github/copilot-instructions.md`

Day 90 · 8月11日（周二）— 性能测试

- `hey -n 500 -c 50 https://localhost:8080/api/dashboard` 并发压测，目标：P99 < 3 秒
- 账单批量生成 639 条压测：`time curl -X POST /api/invoices/generate`，目标 < 30 秒
- PostgreSQL 慢查询分析：`pg_stat_statements` 找出 Top 5 慢查询 + `EXPLAIN ANALYZE` 优化
- 浏览器 Performance / Vue DevTools：Dashboard 页首屏渲染检查

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：
>
> 执行性能测试并生成报告 `docs/qa/performance_test_report.md`。测试步骤：① `hey -n 500 -c 50 -H "Authorization: Bearer {token}" http://localhost:8080/api/noi/summary` 记录 P50/P95/P99（目标 P99 < 3000ms）；② `time curl -s -X POST -H "Authorization: Bearer {manager_token}" -H "Content-Type: application/json" -d '{"month":"2026-08"}' http://localhost:8080/api/invoices/generate` 记录实际耗时；③ `SELECT query, calls, mean_exec_time FROM pg_stat_statements ORDER BY mean_exec_time DESC LIMIT 5` 找出极慢查询；④对最慢查询跑 `EXPLAIN (ANALYZE, BUFFERS)` 分析索引命中情况，若 `Seq Scan` 替换为 `Index Scan` 可减少超 10 倍扫描行数，则生成 `CREATE INDEX CONCURRENTLY` 迁移 SQL。报告中记录每项实际数值，**高亮未达标项并附优化方案**。
>
> 附：`@file:.github/copilot-instructions.md` `@file:docs/ARCH.md`（性能目标：P99 < 3s、账单 < 30s）

Day 91 · 8月12日（周三）— 安全审查

- SQL 注入：全库 grep 确认所有 SQL 使用参数化查询（无字符串拼接）
- JWT：确认算法固定为 HS256（`only: ['HS256']` 校验，禁止 `alg: none`）
- IDOR：所有 `GET /api/xxx/:id` 端点测试跨用户访问是否 403
- 证件号：grep 全库确认 API 响应中 `idNumber` 均脱敏（仅末 4 位），数据库字段均有 `// encrypted` 注释
- 二房东隔离：行级隔离 SQL 条件覆盖测试（携带非授权 contractId，期望空结果 / 403）
- CORS：生产环境 `CORS_ORIGINS` 限制为实际域名（非 `*`）

> 💬 **Copilot 提示语**（模板：`/security-and-test`）：

> 执行完整安全审查并生成报告 `docs/security/security_audit_report.md` 。逐项执行：①SQL注入扫描： `grep -rn 'execute\|query' backend/lib --include="*.dart" | grep -v '\$[0-9]' | grep -v '//'` 输出可疑行（期望零条）；②JWT 算法： `grep -rn "alg\|algorithm\|none" backend/lib --include="*.dart"` 确认无 `alg:none` ；③IDOR：对每个 `GET /api/{resource}/:id` 端点，用另一个用户 JWT 访问，期望 403 或无越权数据；④idNumber 脱敏： `grep -rn "idNumber\|id_number" backend/lib --include="*.dart" | grep -v MaskingUtils | grep -v "// encrypted"` 期望零裸露字段；⑤CORS：验证 `backend/lib/config/app_config.dart` 中 `corsOrigins` 在生产模式下从 `CORS_ORIGINS` 环境变量读取（不含 `*` ）。每项结果记录 Pass/Fail + 证据截图。
>
> 附： `@file:.github/copilot-instructions.md` （安全要求：证件号加密、JWT HS256、行级隔离） `@file:docs/ARCH.md`

Day 92 · 8月13日（周四）— 回归缺陷修复

- 修复 Day 89-91 测试发现的 P0 / P1 缺陷（P0 = 崩溃/数据错误，P1 = 功能不符合 PRD）
- P2 级问题（UI 细节 / 非核心功能）记录到 Backlog，不阻断上线
- 审计日志最终完整性检查（4 类操作覆盖：合同变更 / 账单核销 / 权限变更 / 二房东数据提交）

> 💬 **Copilot 提示语：**
>
> 修复 Day 89-91 发现的所有 P0/P1 缺陷。修复优先级：P0（系统崩溃 / 数据计算错误 / 安全漏洞）必须当日修复且回归验证；P1（功能不符合 PRD / UI 无法操作）当日修复或明日上午前完成。P2（UI 微调 / 非关键功能缺失）记录到 `docs/qa/backlog.md` 并标注"Phase 2 处理"，**不因 P2 延误上线计划**。审计日志最终校验： `SELECT DISTINCT operation FROM audit_logs` 查看覆盖的操作类型，确认包含 `contract_update` 、 `payment_reconciliation` 、 `permission_change` 、 `sublease_review` 四类（缺失则检查对应 Service 是否漏写了 audit log 调用）。修复每个 bug 后在 `docs/qa/regression_test_matrix.md` 对应行更新状态为 ✅ `Pass` 。
>
> 附： `@file:.github/copilot-instructions.md` （审计日志要求：4类操作必须覆盖） `@file:docs/PRD.md`

Day 93 · 8月14日（周五）— 部署准备

- 编写 `README.md` （启动步骤 / 环境变量 / 数据库初始化 SQL 运行顺序）
- `.env.example` （6 个必填 + 3 个可选变量，含说明注释）
- `Dockerfile` （多阶段构建：build → dart compile AOT → 最小 runtime 镜像）
- 数据库自动备份脚本（每日 pg_dump + 7 天滚动删除）
- 生产环境 Checklist（防止上线遗漏）

> 💬 **Copilot 提示语：**
>
> 准备部署文件。 `README.md` 必须包含：①6 个必填环境变量清单（ `DATABASE_URL` 、 `JWT_SECRET` 、 `JWT_EXPIRES_IN_HOURS` 、 `FILE_STORAGE_PATH` 、 `ENCRYPTION_KEY` 、 `APP_PORT` ）及说明；②数据库初始化顺序（ `001_init.sql → 002_seed_kpi_metrics.sql → ...` ，不跳步骤）；③前端构建命令 `cd app && npm run build` / `cd admin && npm run build` 。 `.env.example` 每行含注释（ `# 说明` ），JWT*SECRET* 旁注明 `# 最

> 少 32 个字符`，**不在 `.env.example` 中放真实密钥**。`Dockerfile` 多阶段构建：*Stage 1* 基于 `dart:stable` AOT 编译（`dart compile exe`），*Stage 2* 基于 `debian:bookworm-slim`（不用 dart:stable 作 runtime，减小镜像体积）。生产环境 *Checklist* 保存为 `docs/ops/production`launch_checklist.md （检查项含：所有环境变量已配置 / 数据库已备份 / `CORS` 非 `*` / HTTPS 证书有效）。
>
> 附：`@file:.github/copilot-instructions.md` （必填环境变量清单、安全规范） `@file:docs/ARCH.md`
>
> **Milestone 6**：全部 PRD 验收项 Pass；50 并发压测 P99 < 3 秒；实际 639 套数据导入完成；安全审查无 P0 漏洞。

Phase 7：用户验收测试（UAT）+ 正式上线

> 时间：2026-08-17（W20）— 2026-08-21（周五），共 5 个工作日

Day 94 · 8月17日（周一）— UAT 启动 + 资产模块演示

- 向超级管理员/运营管理层演示 M1 资产台账、楼层热区图操作
- 采集用户反馈（UI 易用性、数据显示、操作流程），填写 UAT 问题单
- M1 发现问题即时修复（当天修复当天验证）

> 💬 **Copilot 提示语：**
>
> UAT 第一天：M1 资产台账与楼层热区演示。使用生产等量的真实导入数据（639 套），**不使用测试 Mock 数据**。演示前检查：①楼层热区颜色与合同到期日一致（随机抽查 3 套单元，对比热区色块与 `GET /api/units/:id` 返回的 `daysUntilExpiry` ）；
> ② `leased` / `expiring_soon` / `vacant` / `non_leasable` 四种状态色使用的是 CSS 变量 `--color-success/--color-warning/--color-danger/--color-neutral` （或 Element Plus `type="success/warning/danger/info"` ），**不是硬编码 hex**。UAT 问题单格式保存到 `docs/qa/uat_feedback.md` （表格列： `| 功能点 | 反馈描述 | 优先级P0/P1/P2 | 处理方案 | 负责人 |` ）。当场发现 P0（崩溃/数据错误）立即修复后重演示；P1 UI 问题记录后**继续演示，不中断 UAT**。
>
> 附： `@file:docs/PRD.md` （§七 验收标准） `@file:.github/copilot-instructions.md`

Day 95 · 8月18日（周二）— UAT 合同 + 工单模块

- 租务专员操作 M2（新建合同 / 录入递增规则 / 查看 WALE 仪表盘）
- 前线员工操作 M4 移动端报修（扫码 / 手填 / 上传照片 / 查看进度）全流程演示
- 二房东账号操作外部填报门户（登录 / 填报子租赁 / 提交审核）

> 💬 **Copilot 提示语：**
>
> UAT 第二天：M2 合同模块 + M4 工单移动端 + M5 外部门户。M2 演示要点：①新建合同时录入含 2 段递增的混合规则，保存后进入合同详情页确认 `escalationRules` JSONB 正确展示；②查看 WALE 仪表盘，数值与预期年数对应（WALE 公式： `Σ(剩余租期ᵢ × 年化租金ᵢ) / Σ(年化租金ᵢ)` ）。M4 演示要点：使用真机扫生成的 QR 码（ `/api/qr/unit/:id` ），确认直接跳转到报修提交页（**不需要手动输入房号**）；工单提交后在运维人员端 App 收到任务推送（推送降级 → `/api/alerts` 兜底机制也须演示）。M5 演示要点：二房东使用独立账号登录，**仅能看到自己名下子租赁数据**，访问其他二房东编号返回 403（演示 `GET /api/sub-`

> `leases?landlordId=other` 返回空集或 403）。问题记录到 `docs/qa/uat_feedback.md`，与 Day 94 同表格追加。
>
> 附：`@file:docs/PRD.md`（§七 验收标准）`@file:.github/copilot-instructions.md`

Day 96 · 8月19日（周三）— UAT 财务 + KPI

- 财务人员操作账单核销、录入支出、查看 NOI 看板
- 管理层查看 KPI 评分仪表盘（雷达图 / 排名榜 / 历史趋势）
- UAT 问题单汇总，确认上线阻断项

> 💬 **Copilot 提示语：**
>
> UAT 第三天：M3 财务 + KPI 演示。财务演示要点：①批量账单生成（演示 `POST /api/invoices/generate`，显示已生成条数和金额合计）；②核销一笔账单（状态从 `pending` → `paid`，确认列表状态标签变为 `type="success"` 绿色）；③NOI 看板显示公式说明 `NOI = EGI - OpEx`，数字可与财务手工计算对比。KPI 演示要点：查看当月 KPI 评分仪表盘，雷达图 10 个指标均有数值（满意度指标为手动录入，其余 9 个自动，空值显示 **0** 分而非报错）；尝试提交 KPI 申诉（提交后状态变 `appealing`，管理层端显示待审申诉提醒）。UAT 问题汇总后，在 `docs/qa/uat_feedback.md` 末尾添加"上线阻断项清单"区块，标明 P0/P1 问题数量及全部确认"已修复"方可批准上线。
>
> 附：`@file:docs/PRD.md`（§七 验收标准）`@file:.github/copilot-instructions.md`

Day 97 · 8月20日（周四）— UAT 缺陷修复

- 修复 UAT 反馈的全部上线阻断问题（P0/P1 级，限制在 6 小时内）
- 更新操作手册截图（若 UI 有调整）
- 验收签字确认（与运营团队负责人确认验收通过）

> 💬 **Copilot 提示语：**
>
> UAT 最终缺陷修复日。修复规则：P0（崩溃/数据错误/安全漏洞）和 P1（核心功能缺失/错误数据展示）必须在今天 6 小时内全部修复并复测通过；P2（样式细节/非核心体验）允许推迟到 Phase 2。修复后每个问题在 `docs/qa/uat_feedback.md` 标记 ✅ 已修复（含修复 commit hash 便于追溯）。修复时严格检查四类操作的审计日志是否仍完整（`SELECT DISTINCT operation FROM audit_logs` 结果必须包含：`contract_change` / `invoice_write_off` / `permission_change` / `sub_landlord_submit`）。若今日 18:00 前 P0/P1 全部修复且复测通过，运营负责人可签字验收；否则上线推迟一个工作日，不得强行上线。操作手册截图更新至 `docs/ops/user_manual_screenshots/`（按模块子目录存放）。
>
> 附：`@file:docs/PRD.md`（§七 验收标准）`@file:.github/copilot-instructions.md`

*Day 98 · 8月21日（周五）— **正式上线***

- 生产环境部署：后端服务启动验证（6 个必填环境变量 assert 通过）、PostgreSQL 连接、文件存储挂载
- 生产环境冒烟测试（登录 / 数据查询 / 账单生成各触发一次）
- 全量数据在生产库确认完整（与测试环境数据核对关键指标）
- 向团队宣布 **PropOS Phase 1** 正式上线
- 开始 Phase 2 规划冲刺（租户自助门户 / 电子签章优先级评估）

> 💬 **Copilot 提示语：**
>
> 正式上线日 — 严格按顺序执行。①服务启动验证：6 个必填环境变量（`DATABASE_URL` / `JWT_SECRET` / `JWT_EXPIRES_IN_HOURS` / `FILE_STORAGE_PATH` / `ENCRYPTION_KEY` / `APP_PORT`）缺失时服务必须 panic 退出并输出明确错误（`app_config.dart` 已有 assert，验证其在生产日志中输出 ✅ `All required env vars present`）。②健康检查：`GET /api/health` 返回 `{"status":"ok","db":"connected"}`（不含任何内部路径或版本号敏感信息）。③冒烟测试三步：`POST /api/auth/login`（正常返回 JWT）→ `GET /api/units?page=1&pageSize=20`（返回 meta.total = 639）→ `POST /api/invoices/generate`（当月，返回 successCount > 0）。④生产数据核查：`SELECT count(*) FROM units` = 639，`SELECT count(*) FROM contracts WHERE status='active'` 与测试环境对比无差异。⑤上线通知：向团队发布 PropOS Phase 1 正式上线公告，附 Phase 2 优先级评估启动计划。**上线后第一时间通知运维人员保存当前 pg_dump 快照作为 Day-0 基准备份。**
>
> 附：`@file:.github/copilot-instructions.md`（必填环境变量、安全规范）`@file:docs/ARCH.md`（Phase 2 功能范围边界）
>
> **最终里程碑：PropOS Phase 1 于 2026-08-21 正式上线，639 套房源三业态全数字化管理就位，M1~M5 五個模块全面运行。**

关键风险预案

风险点	概率	影响	预案
CAD .dwg → SVG 转换工具不可用或输出质量差	中	M1 延期 1–2 天	W3 末若 ODA File Converter + ezdxf 链路输出质量不符合要求，降级为"手动上传 SVG/PNG"；CAD 自动转换列为 Phase 2 补充
租金递增配置器 UI 复杂度超估	中	M2 延期 2–3 天	Day 36 前若配置器 UI 未完工，先简化为纯 Form 表单（暂不含实时预测图），功能完整性优先
KPI 10 指标跨模块聚合 SQL 难度超估	中	M3 延期 2 天	K10 满意度固定为手动录入；其余 9 个自动指标若有 2 个数据源不稳定，先返回 0 值，待稳定后补接
真实 CAD 文件结构与测试文件不同	中	数据初始化延期	Day 13 前向运营团队索取 1 层实际 .dwg 文件用作早期验证
外部门户行级隔离漏洞	低	严重安全问题	M5 行级隔离 SQL 必须先于外部门户联调完成；Day 91 安全审查作为硬性上线 Gate
数据初始化工作量超出预期	中	Phase 6 延期 1 周	提前 2 周与运营团队划定责任边界：数据收集/整理由团队负责，导入工具由开发方提供
功能范围蔓延（Scope Creep）	高	整体延期	Phase 2 功能请求（租户门户 / 门禁 / 电子签章）严格挡在上线后评估；Copilot 指令中已明确禁止超前实现

附录：工作日历

> 中国法定节假日影响（2026 年）：
>
> - 五一劳动节（5/1–5/5，本计划 5/1 已纳入工作日，如需休假则 M1 顺延一天）
> - 端午节（5/29 前后，如为法定休假则 M2 调整 1 天）
> - 国庆长假（10月，Phase 1 在 8月完成，不受影响）

建议：节假日对应的工作内容可在前后 1 天内灵活调整，不改变里程碑日期目标。

本计划基于 *PropOS PRD v1.8*（*2026-04-09*）及 *ARCH.md v1.4*（*2026-04-09*）制定。如 *PRD* 需求变更，相关模块排期对应调整。

附录 B：v1.7 需求变更对排期的影响评估

> 以下变更项来自 PRD v1.6 → v1.7 升级，均为 Phase 1 Must 范围内新增能力。原 v1.0 日程计划的任务骨架保持不变，但涉及以下模块的工作日需相应扩展。

新增工作量估算

变更项	影响阶段	估算新增工作日	建议嵌入时段
合同-单元 M:N 关联（contract_units）	Phase 2 M2	+1 天	Day 25~26 合同数据层
合同提前终止流程（3 种终止类型）	Phase 2 M2	+1 天	Day 26~27 状态机
含税/不含税标识 + 税率字段	Phase 2 M2 + Phase 4 M3	+0.5 天	Day 25（后端）+ Day 65（前端）
押金独立管理（deposits + transactions）	Phase 2 M2	+2 天	Day 27~28 之间新增
租户信用评级（A/B/C 自动重算）	Phase 2 M2	+0.5 天	Day 24 Tenant 后端
水电抄表与计费（meter_readings）	Phase 4 M3	+2 天	Day 55~56 之间新增
商铺营业额申报审核（turnover_reports）	Phase 4 M3	+1.5 天	Day 39（已有骨架）扩展
WALE 双口径（收入加权 + 面积加权）	Phase 2 M2	+0.5 天	Day 28 WALE 服务
KPI 指标方向（positive/negative）	Phase 4 M3	+0.5 天	Day 59~61 KPI 引擎
导入批次管理（import*batches* + *dry*run + 回滚）	Phase 1 M1 + Phase 6	+1 天	Day 13 导入后端
迁移脚本扩展（10 步 → 14 步）	Phase 0	+0.5 天	Day 4~5
PIPL 合规 + HTTPS/TLS + 密码复杂度	Phase 5 M5 + 安全审查	+0.5 天	Day 91 安全审查
合计		**+11** 天	

排期调整建议

1. **不改变里程碑日期**：通过提高并行度和减少缓冲来吸收新增 11 天工作量。具步骤：
   - Phase 2（M2）原计划 20 天缓冲 2 天 → 改为满载；押金管理和合同终止与合同 CRUD 交叉推进。
   - Phase 4（M3）原计划 20 天 → 水电抄表与账单生成并行后端开发；营业额申报复用 Day 39 已有分成代码骨架。
   - Phase 6（集成测试）原计划 10 天 → 将导入批次测试与数据初始化合并执行。
1. **若无法吸收，备选方案**：将 S-06（KPI 试运行看板）和 S-07（信用评级可视化）从 Should 延后至 Phase 1.5，释放约 3 天给 Must 新增项。
1. **风险提示**：新增 4 张数据表（deposits、meter*readings*、*turnover*reports、import_batches）及对应 API/页面，开发密度上升约 12%。建议在 Wave 2 结束时（Day 43 前后）评估进度，若落后超 3 天则立即启动 Should 延期决策。

---

v1.2 对齐 data_model v1.3 补充说明（2026-04-08）

以下 data_model v1.3 新增项在上方日计划中未显式提及，开发时需注意在对应 Day 中一并实现：

data_model v1.3 新增项	影响 Day	具体补充
`floor_plans` 多版本图纸管理（`is_current` 标识当前生效）	Day 10（Building + Floor 后端）	`FloorRepository` 需增加 `floor_plans` 表 CRUD + `setCurrentPlan` 方法
`escalation_templates` 递增规则模板保存/应用	Day 26（递增规则持久化）	需增加 `EscalationTemplateRepository` 与 `POST /api/contracts/:id/apply-template`
`alerts.target_user_id` 定向推送	Day 29（预警引擎后端）	`alert_service.dart` 生成预警时需填充 `target_user_id`，支持用户级定向
`data_retention_until` PIPL 合规	Day 24（租客后端）+ Day 49（二房东后端）	`tenants` 和 `subleases` 表新增字段，合同终止后计算保留截止日
`contracts.status` 默认值 `quoting`	Day 25（合同 CRUD 后端）	创建合同时状态初始化为 `quoting` 而非 `pending_sign`

---

v1.4 对齐 PRD v1.8 / ARCH v1.4（2026-04-13）

以下 PRD v1.8 新增项在上方日计划中未显式提及，开发时需注意在对应 Day 中一并实现：

PRD v1.8 新增项	影响 Day	具体补充
`expense_category` 枚举新增 `professional_service`（专业服务费）	Day 4（001*create*enums.sql）	迁移时补充枚举值；`expense` 录入页UI新增「专业服务费」选项
`work_orders.cost_nature` 列（opex/capex）	Day 38（工单完工流程）	`006_create_workorders.sql` 新增 `cost_nature` 列（NULLABLE，仅 repair 类型）；完工API请求体增加 `cost_nature` 字段

`noi_budgets` 表（NOI 年度预算）	Day 55（NOI 看板后端）	`018_add_noi_budgets.sql` 新建；`NOIBudgetRepository` / `NOIBudgetService` + `GET/POST /api/noi/budget`
NOI Margin/OpEx Ratio 指标聚合	Day 55（NOI API）	`GET /api/noi/summary` 响应增加 `noi_margin` 和 `opex_ratio` 字段
NOI 看板预算达成率（K07）	Day 55（NOI 看板前端）	前端看板增加"预算 vs 实际"对比卡片，调用 `GET /api/noi/budget`